

Aristotle University of Thessaloniki
School of informatics

# Pooling Compute across Machines

A Rudimentary System for Batch Job Processing

Theodoros Kaimasidis

Supervisor: Petros Nicopolitidis

October 15, 2026

# Ευχαριστίες

Θα ήθελα να ευχαριστήσω την κοπέλα μου Έλλη Γ. και την οικογένειά μου για τη στήριξη, καθώς και τον επιβλέποντά μου κ. Πέτρο Νικοπολιτίδη για την ελευθερία να ασχοληθώ με τόσα από τα θέματα που με ενδιαφέρουν.

# Abstract

This work is trying to solve a fundamental issue, that of combining many computing systems in order to create one of larger capacity, without however creating something that is embarrassingly-parallel. The exposure to the underlying mechanisms of the systems involved is also a welcome challenge, and with this I want to distill experience that would be hard to amass in a professional setting with the exciting field of research in the area of massive computation. Inspiration is drawn from the distributed systems applicable to commercial applications and software, as well as the more scientific endpoints of linux and the fundamental technologies used. The result resembles a RPC framework, with reusable and extensible components that for the most part bond well to the underlying structure of the orchestrator that manages the lifecycles of the userspace programs we are going to be using to have run the set of computations, of which results we are goingto base ourselves on for the evaluation.

# Contents

**Ευχαριστίες** ι

**Abstract** ii

**1 Introduction** 1
1.1 How can we combine two or more computers to accelerate computation of one or multiple tasks? . . . 1
1.2 The hard part . . . 1
1.3 Structure of this Thesis . . . 2
1.3.1 Use of AI tools . . . 2

**2 Containers, Orchestration, and Scaling** 3
2.1 Docker and the container model . . . 3
2.1.1 Images . . . 3
2.1.2 Mounts . . . 4
2.1.3 Networks . . . 4
2.1.4 Configuration and Compose . . . 5
2.1.5 Isolation . . . 5
2.1.6 PID 1 and shutdown . . . 5
2.2 Kubernetes: orchestration and the control plane . . . 6
2.2.1 kubectl and kube-apiserver . . . 7
2.2.2 etcd . . . 8
2.2.3 kube-scheduler . . . 8
2.2.4 Cluster Autoscaler / Karpenter . . . 8
2.3 Kubernetes: the worker node . . . 9
2.3.1 kubelet . . . 9
2.3.2 Container Runtime . . . 10
2.3.3 kube-proxy . . . 10
2.4 Kubernetes: the object kinds . . . 10
2.4.1 How the Kinds Relate . . . 10
2.4.2 Pod . . . 11
2.4.3 ReplicaSet . . . 11
2.4.4 Deployment . . . 12
2.4.5 StatefulSet . . . 13
2.4.6 DaemonSet . . . 13
2.4.7 Job . . . 14
2.4.8 CronJob . . . 15
2.4.9 Service . . . 15
2.4.10 ConfigMap & Secret . . . 16
2.4.11 The object model, and where each kind fits the engine . . . 16
2.5 API gateways, ingress, and service mesh . . . 17

2.6 The engine on Kubernetes: which entities it uses, and for what . . . 17
2.6.1 Pod: one pod is one engine node . . . 18
2.6.2 StatefulSet rather than Deployment . . . 19
2.6.3 Service: headless, for naming only . . . 20
2.6.4 Job: a batch, submitted and gone . . . 20
2.6.5 ConfigMap and the downward API . . . 21
2.6.6 Autoscaling: KEDA, the HPA and metrics-server . . . 21
2.6.7 Scheduler, kubelet and runtime, and the network . . . 21
2.6.8 What is deliberately not used . . . 22
2.6.9 An alternative split: one head pod, many worker pods . . . 22

**3 The Engine** **24**
3.1 The high level design . . . 24
3.2 The job model . . . 24
3.3 One wire format . . . 25
3.3.1 In `engineJava`: from task frames to objects . . . 26
3.4 Identity: ids computed from the bytes . . . 26
3.5 DAG execution . . . 27
3.6 Dependency tracking without consensus . . . 28
3.7 Shipping code . . . 28
3.8 Deployment . . . 28
3.9 The `engineJava` node . . . 29
3.9.1 The worker . . . 29
3.9.2 The filesystem as the reply path . . . 29
3.9.3 The head . . . 29
3.10 The cluster layer . . . 30
3.11 What was run and measured . . . 30
3.11.1 Bare nodes against the deployment . . . 31
3.12 Considered and not used . . . 31
3.13 What the engine demonstrates, and what it does not . . . 31
3.14 Two machines over one 1 Gb link . . . 32
3.14.1 The setup . . . 32
3.14.2 Running it . . . 33
3.14.3 The result . . . 33
3.14.4 The same load over two 10 Gb links . . . 34

**4 Enhancements** **36**
4.1 XDP: programmable packet processing . . . 36
4.1.1 The idea . . . 36
4.1.2 eBPF: how the program gets into the kernel safely . . . 37
4.1.3 Hello, XDP . . . 37
4.1.4 Attachment modes and AF_XDP . . . 38
4.1.5 More usages of XDP for the system . . . 39
4.1.6 Evaluation of performance gains . . . 41
4.2 Protocol Buffers . . . 43
4.2.1 Hello, protobuf . . . 43
4.2.2 The 19 bytes . . . 44
4.2.3 Schema evolution . . . 44
4.2.4 How schema-based encodings work: Protocol Buffers, Thrift and Avro . . . 44
4.2.5 Framing, and where protobuf fits . . . 45
4.2.6 Where it would fit in the engine . . . 45
4.3 IPC: the controller's mechanisms . . . 45

4.3.1 Processes, signals and the lifecycle . . . 46
4.3.2 Threads as workers . . . 46
4.3.3 The IPC mechanisms . . . 46
4.3.4 Configuring the channels . . . 47
4.3.5 Controllers and client–server pairs . . . 47
4.3.6 The worker's `main` . . . 47
4.3.7 The biggest gain to be had . . . 47
4.3.8 XDP and io_uring, together . . . 48
4.4 KEDA: scaling on the engine's own load . . . 49
4.5 CRIU: moving a process instead of losing it . . . 50

# Chapter 1

# Introduction

This thesis is about computation, systems, distributed computation and their interwining in an interesting project. The need is accelerated computation via the means of one or more computers/computation nodes, which may have lost some of its importance in our days, thanks to many available multi-core consumer and server computer processors, but it's one aspect of computing systems that's extremely rewarding and has too much to teach in the process of its implementation. The problem visited is a personal question of mine from when I was way younger, how can I combine two slow machines to run something faster.

## 1.1 How can we combine two or more computers to accelerate computation of one or multiple tasks?

This is something that many pc enthusiasts dream of doing, combining the power of multiple cpus or computers for more compute, and it's actually been a huge trend with the advert of data centers and distributed systems evolution and adoption on the business of software development and deployment. The case reviewed here is one about normal pcs with clear boundaries and store bought interconnects (Ethernet) facilitating their intercommunication. The first challenge met is the design of the system, unlike the 'system-design' paradigm of today's development circles however the architecture here can scale up and down, traverse and live in different layers. For example computation is discussed in the context of an os, also in the process context of the C runtime and also with the linux operating system being open source we have access to both with one tool: the C programming language. Further, concurrency is a "solved" issue, the problem is the communication and mutual exclusion of what constitutes a thread in our application, it's been a struggle designing something that is not a mere "embarrassing-parallel" machine, while it's been a similar note for the application of the distributed systems side of algorithms for consensus and communication between nodes.

## 1.2 The hard part

The hard part comes before the implementation, the decision of what to focus on, what to work on, what to reuse and what to write ourselves. The result of this work is software that can run on two or more machines and via a gateway obfuscate the raw computing power of both to provide higher compute than each of the constituents, hopefully in a scalable way (and in a scalable way that makes sense). For a start we need to define the "computation", what exactly is going to be worked up on: the choice has been made for this to be a set of input data and a function that's to run on them. This comes instead of a process, which in linux is completely open to a developer to see, even if there needs to be a kernel loadable module. Furthermore, this pair constitutes a "job", and the objective is to be able to distribute them in the environments that we control. A

plain worker written in C or Java for example would not suffice here, it's clear that there is a need of a head and a worker component on each installation. As a result after the choice of the execution mechanism, what is needed is a routing algorithm, or a choice of a routing strategy in order to be able to provide an actual higher capacity for computations across two systems, which in our finalized project is taken care of the head (written in C) that constitutes a distributed system. And finally, the landing ground for these has been set to be the kubernetes orchestrator engine, which means packaging in containers and managing of the compute nodes accordingly, this choice has been made because the philosophy of this technology perfectly contributes to the idea of the pooling together of the resources of multiple computers and also is in and of itself an interesting endeavor since much of the modern infrastructure operates in the form of a kubernetes cluster. What has been clear from the start of the project however is the same as what striked me most when I first studied concurrency, running threads in parallel is easy, initiating mulitple processes is also easy, problems arise when we have exclusivity and in this way bottlenecks.

## 1.3 Structure of this Thesis

This thesis puts into context the software project that is the showcase of the work. In total it explains the technologies used, explores the underlying concepts and the mechanisms that are exposed via apis or the ones that are applicable to the project in a meaningful way. A more general note of exploration and care has been given because the whole project is a reflection of my own personal interests.

In so doing, the chapters move from the platform the engine runs on to the engine itself, and then to what could improve it.

Chapter 2 covers Docker and Kubernetes: the container model, the control plane, the worker node and the object kinds, with Karpenter for "compute node" discovery, and which of these entities the engine uses when it is deployed on a cluster.

Chapter 3 describes the engine: the design decisions of the first prototype, and `engineJava`, which kept them and made the engine distributed, as it is built and measured.

Chapter 4 covers the enhancements: XDP, Protocol Buffers, inter-process communication for a controller of the engine's own (with XDP and io_uring), KEDA and CRIU, and a short section on the consensus algorithms the engine relies on without implementing.

### 1.3.1 Use of AI tools

I state that this work is my own. AI was used as an assisting tool in various stages of the project, but it did not determine the project's core ideas, design, or content.

For the **code**, all major design decisions were my own, including the design of the hexagonal architecture of the "head" and the worker. AI was used primarily as a development aid for adding error logging, debugging, performing quick consistency checks on the "wire", and general brainstorming.

For the **text and documentation**, AI was seldom used for the actual writing. It was mainly used to help parse and structure material into LaTeX, compile the accompanying presentation, and occasionally provide recaps of books and assistance with the bibliography. It was also used to keep track of bibliographic sources used during the project.

The intellectual content, design decisions, implementation, and final work remain my own.

# Chapter 2

# Containers, Orchestration, and Scaling

The engine of Chapter 3 is correct but static: a fixed set of worker processes started by hand. To make it scale effortlessly — to add capacity by declaring a number rather than by wiring up machines — its workers must become a fleet that some external system starts, replaces, places, and grows automatically. That system is a container orchestrator [2, 4], and in practice it is Kubernetes running containers built with Docker/OCI tooling. This chapter is a thorough, advanced introduction to both, oriented throughout toward one payoff: run behind an orchestrator, the coordinator/worker engine turns into a *horizontally elastic function service* — tasks submitted, scheduled onto an autoscaled worker pool, and executed on demand. Structurally that is a self-hosted *distributed lambda*, and when tasks form a map/shuffle/reduce DAG it is a *MapReduce*. The recurring theme is a boundary: what the orchestrator may own versus what the engine must never delegate [3].

## 2.1 Docker and the container model

*In one line: a container is a resource-capped, namespaced process — these are enough to simulate an effectively isolated environment.*

Historically we could say that containers are the continuation or adaptation of virtual machines, but this is only true in the way that virtual machines are the replacements of actual server machinery. Containers are however the last executable and writable layer on top of an image, in which all the "work" is distilled, and the biggest difference with virtual machines is that containers rely on the host os to run, that is the host kernel, making it easy to have multiple containers in one computer run, and with the isolation mechanisms having been developed them taking turns executing is under complete isolation from each other. Images need to be created by dedicated teams collecting all application dependencies, and it needs to be in need only of one kernel in order to run, because all programs that are to run on the container will be completely separated from the outside world. Otherwise it is an ordinary Linux process (or process tree) started with a restricted view on the host system.

### 2.1.1 Images

**What it is:** The most important noun is for sure the *image*, which is the read-only basis of the container, just like the java source is the blueprint of a class in which case the container would be an instance of it - an object -, in actuality it would be a running instance of an image with the container being a writable layer on top of it.

**What the daemon manages:** Images package the filesystem and metadata, and stand to containers as a class stands to an object. An OCI image is an ordered set of tarball *layers* unioned at runtime (overlayfs) into a root filesystem, plus a JSON config (entrypoint, env, exposed ports). The daemon does the stacking; what the container sees is a single tree, with one thin writable

layer on top that dies with it. Layers are content-addressed, so identical layers are cached and shared across images and pulled once.

**The practical discipline:**

- order the Dockerfile so rarely-changing layers (dependencies) come before volatile ones (your binary);
- use *multi-stage* builds to keep compilers out of the runtime image;
- ship a *distroless* or `scratch` base — for the statically linked C worker of this project, a few megabytes with almost no attack surface;
- reference images by *digest* (`image@sha256:...`) in production, never by mutable tag, so a redeploy cannot silently change the binary.

### 2.1.2 Mounts

**What it is:** What needs to be persisted needs to be attached to the container by mount. Persistent state arrives in one of three ways:

- **Volume** — a named volume managed by the daemon. The volume (directory) needs to be mount on the container's filesystem.
- **Bind mount** — maps a specific host directory into the container instead, which also facilitates perfectly development needs.
- **tmpfs** — for secrets that should never reach a layer.

None of it is distributed — two cells on two hosts sharing a volume means a network filesystem, with all of its consistency behaviour intact.

### 2.1.3 Networks

**What it is:** Communication to the outside world happens via the docker network drivers. Each container receives its own network namespace and a virtual interface whose far end sits on a host bridge, with routing between them and NAT outward; a published port is a rule mapping a host port inward, and it is the one place the isolation is deliberately opened. That default model puts every container on one L2 segment, which is the structural reason inter-container traffic needs separating rather than trusting.

Containers on Docker's default `bridge` network can reach the outside world, but container-to-container communication does not provide the convenient name-based discovery available on user-defined networks. Consequently, the first practical setup step is almost always to create a user-defined network, within which containers can refer to one another by name, with Docker providing the necessary DNS resolution.

**Drivers:** Docker networks are implemented through different network drivers, each corresponding to a different networking model and use case. The most important drivers are:

- `bridge` — the normal choice when several containers run on the same Docker host. Docker creates a virtual network bridge on the host, and containers attached to it receive their own network interfaces and IP addresses. A user-defined bridge is preferable to Docker's automatically created default bridge because it provides automatic DNS-based service discovery and gives the administrator explicit control over which containers belong to the network.
- `host` — removes much of the isolation of Docker's network. Instead of giving the container its own isolated network stack, the container uses the host's network namespace directly.
- `none` — provides essentially no network connectivity to the container, which makes it useful for workloads that do not require networking and where network exposure should be minimized.

- `overlay` — becomes important when containers are distributed across multiple Docker hosts, allowing them to communicate over a shared logical network spanning those hosts.
- `macvlan` — takes a different approach: instead of making the container appear as an endpoint behind Docker's virtual networking, it can give the container its own MAC address and make it appear as a device directly connected to the physical network. This is useful for applications that need to participate directly in an existing Layer-2 network.

### 2.1.4 Configuration and Compose

Configuration enters through environment variables (and, for anything sensitive, files mounted from tmpfs rather than baked into the image, since image layers are readable by anyone who can pull them). In practice these four are created/defined in a Compose file that declares the images, the volumes, the networks and the published ports for a whole set of services in one place, and brings them up and down together, even defining dependencies between containers.

### 2.1.5 Isolation

The most important attribute of the container, and the mechanism by which it can be used and considered an analog to a virtual machine is the mechanisms by which we can enforce it's isolation: Namespaces restrict what a process can see (PID, mount, network, IPC, UTS, user, and cgroup namespaces, created by clone with the `CLONE_NEW*` flags), cgroupsv2 limit what it can consume (CPU, memory, I/O, and PIDs), seccomp filters restrict which system calls it may make, and dropped capabilities strip privileges otherwise associated with root, which is a main point of podman, which unlike docker, is daemonless and supports rootless containers, allowing a container to run as root internally while that root identity is mapped through a user namespace to an unprivileged host user, thereby avoiding the need for the container runtime to possess host-root privileges, in linux daemons are all root owned. Namespaces are the mechanism employed in kubernetes as well, they create a folder within the isolated processes have visibility off and the outside world is only available through the managing software and the network. These all show that safety is defined in an application context and for that untrusted code needs to run in a VM-based sandbox (gVisor, Firecracker, Kata).

**What the runtime builds.** A container is not a virtual machine. It is an ordinary Linux process (or process tree) started with a restricted view and a bounded appetite, built entirely from ordinary kernel primitives.

- **Construct** — *namespaces* restrict what it can see (PID, mount, network, IPC, UTS, user, cgroup and time, created by `clone` with the `CLONE_NEW*` flags), and *cgroups v2* bound what it can consume (CPU, memory, I/O, PIDs).
- **Constrain** — dropped *capabilities* strip root's powers one by one, a *seccomp* filter allows only the syscalls it actually needs, and a mandatory access control profile (AppArmor's path-based rules, or SELinux's contexts) confines it at the kernel's own boundary.

The distinction is worth holding on to: the first two make the container, the last three make it safe. There is no guest kernel; a container shares the host's, which is why it starts in milliseconds and also why, by default, it is an isolation boundary but not a security boundary against a kernel exploit — untrusted code wants a VM-based sandbox (gVisor, Firecracker, Kata).

### 2.1.6 PID 1 and shutdown

A container's entrypoint runs as PID 1 in its PID namespace, which changes signal semantics: the kernel delivers no default action for signals to PID 1, and PID 1 must reap orphaned children. So the entrypoint must be the real process (via `exec`), or a tiny init, or `SIGTERM` is ignored and

the orchestrator's graceful-stop request is dropped. Correct `SIGTERM` handling — stop accepting work, finish or hand back in-flight tasks, exit — is what turns a worker eviction from a lease-expiry storm into a clean handover, and it is the single highest-value piece of container-awareness the engine needs. Resource limits are cgroup limits: a memory limit yields a deterministic cgroup-OOM kill (a clean, attributable crash), while a CPU limit *throttles* — descheduling the process for the rest of the period — which stalls heartbeats and is the most under-diagnosed cause of false failure detection (watch `cpu.stat`'s `nr_throttled`).

Docker is not one program but three separable concerns, and almost every property of this section — including every security property — depends on which of the three a mechanism belongs to. The tiers are defined by interfaces rather than by one vendor's product, which is why each is independently replaceable.

- **Runtime** — the software that actually runs a container: it talks to the kernel, sets the isolation up, and gets out of the way.
- **Daemon** — sits above it, serving build, start and stop requests and owning the lifecycle of the objects a user names — images, containers, volumes, networks.
- **Orchestrator** — sits above that, deciding where containers run, scheduling them, and reacting to failures.

**The runtime's division of labour.** The Docker CLI talks to a daemon that delegates to `containerd`, which starts a *shim* per container, which invokes `runc`. `runc` sets up the namespaces, cgroups and pivot-root, installs the seccomp filter and capability set, and `execve`s the entrypoint per the OCI runtime spec — and then exits, leaving nothing of itself behind. The shim is what remains: it becomes the container's parent, holds its `STDIN`/`STDOUT` open and reaps it, which is why a container survives a restart of the daemon above it. Because the interface is the runtime spec, the runtime is replaceable (`crun`, `youki`, or a VM-backed implementation), and Kubernetes talks to the same lower layers through the CRI (Container Runtime Interface), so Docker-the-daemon is not required on a cluster — `containerd` suffices.

**The third tier.** Everything above is one host. The orchestrator is the tier that decides which host, keeps the declared number of instances running, and replaces the ones that fail — and because images, networks and volumes are already named, portable objects, it can do so by recreating rather than by moving. That is the subject of the next section, and it is also the boundary of what the container model offers this thesis: relocation at the boundary, not migration of a running computation.

## 2.2 Kubernetes: orchestration and the control plane

Mainly kubernetes is an orchestrator for deploying applications, and is generally the de facto infrastructure for distributed systems built today. Kubernetes is a software to manage and deploy applications, literally an orchestrator, the end product is a cluster that is running inside a network of vms, containers, servers. Primarily comprised of two entities (and their constituents): the control-plane and the nodes (data-plane), the management happens declaratively, the desired state is described, and the control is tasked with evaluating the current state, when something is diverging actions are taken to reconcile that. Usually, user-space code will be packaged as a container, and a container will be wrapped in a pod (the smallest deployable unit), next higher-level controllers are going to assist their function across the cores or machines allocated and specified for operation. Although one mostly resorts to distributed architectures for performance gains and reasons, Kubernetes as a manager prides itself in being scalable, reliable and fault tolerant, which are actually the most important aspects for a reliable system, especially with the availability standards that are at play today. What you are buying when you adopt k8s however is efficiency, velocity in deployments, scaling and an abstraction of your infrastructure, which is

the opposite of technical debt, as Kelsey Hightower notes in his book. Immutability in the context of Kubernetes is the strategic decision of not updating resources in a way that is not explicitly stated and baked in the configuration or creation logic, instances of an application need to be exactly the same, but it's not possible reliably without an automation layer or the separation of development and deployment that we are able to achieve with docker, Kubernetes and the related technologies. The developer needs to provide solutions that are going to be turned on and off once, they should work with no more than their base arguments and environment, for example updating a library could equally well be achieved by logging in the container and installing the new dependency, or by creating a new one with the updated library baked in, kill the old and launch the new one. Declarative configuration is also easier to understand since it's an 'image' of the world it wants to create, rather than an iteration on a starting point. Kubernetes owned apis and objects are the pods, or group thereof, the actual deployable units that are collections of containers, then services like load balancing, naming and discovery, namespaces that provide isolation the same way docker does for containers and ingress objects that constitute a frontend towards multiple microservices. All these have the effect of isolating the machine from the userspace application that's running, having a negating effect on costs.

Below are the most important control plane components explained:

### 2.2.1 kubectl and kube-apiserver

**What it is:** The front door and brain of the control plane. It exposes the Kubernetes API. This is a command-line tool that accesses the kubernetes api, kubectl is just the official image of the client. Typical commands can provide information for the components of a simple cluster like the controller-manager, the scheduler, finally etcd, which is the storage of the cluster.

- **The Only Direct Interface to etcd:** `kube-apiserver` is the only component in the entire cluster that communicates directly with etcd, while every other control plane component (scheduler, controllers) and worker component (kubelet) has to go through the API server. That means that this is not the writer and that after etcd writes the info and metadata about the state it's only readable in one place, this comes with the extremely important advantage of providing one place to enforce concurrency control for readers and writers on the distributed system that's to be managed.
- **Stateless & Scalable:** Because `kube-apiserver` stores no local state (it delegates all state persistence to etcd), you can easily scale it horizontally behind a load balancer for High Availability (HA) - installing more instances of it -.
- **Request Pipeline:** Every request, whether issued via `kubectl` or code, moves through a strict processing pipeline before it is ever persisted, and it takes the form of a verb such as `get`, `list`, `watch`, `create`, `update`, `patch`, or `delete` applied to a resource (or a subresource like `pods/exec` or `scale`).

  1. **Authentication:** Confirms the identity of the caller, using mechanisms like client certificates, bearer tokens, or an external identity provider.
  2. **Authorization:** Checks whether that identity is permitted to perform the requested action, typically enforced through Role-Based Access Control (RBAC).
  3. **Admission Control:** Runs a chain of controllers that can mutate or validate the object on its way in, for example injecting default resource limits or blocking requests that violate security policy.
  4. **Storage:** Finally, the API server writes the resulting state to etcd, persisting the state.

  kubectl is the primary tool for interacting with Kubernetes objects, which are organized into namespaces (folders you can target with –namespace, or view all of at once with –all-namespaces) and accessed through contexts stored in /.kube/config, which let you set a

default namespace and cluster/user combo via kubectl config set-context and use-context. To view objects, kubectl get lists resources (optionally with -o wide, -o json, -o yaml, or JSON-Path queries via -o jsonpath for extracting specific fields), while kubectl describe gives a fuller human-readable rundown including related events. Objects themselves are defined as YAML or JSON files and managed declaratively with kubectl apply -f, which creates or updates only what's changed (previewable with –dry-run) and keeps a history of prior configurations you can inspect with view-last-applied; you can also edit objects interactively with kubectl edit, or remove them with kubectl delete, which executes immediately without confirmation. Finally, labels and annotations — arbitrary tags on objects — are managed with kubectl label and kubectl annotate (requiring –overwrite to replace an existing label, and a trailing - on the label name to remove one).

### 2.2.2 etcd

**What it is:** A strongly consistent, distributed key-value store based on the Raft consensus algorithm, where functions as the storage of all of the api objects.

- **Cluster Storage of state:** Every single object created in Kubernetes (Pods, Services, CRDs, ConfigMaps) lives inside etcd.
- **Watch Mechanism:** Instead of components constantly polling the database (which degrades performance), etcd supports streaming updates. The API server watches etcd and streams real-time events to the controllers/schedulers.
- **Consistency Over Availability:** etcd prioritizes data consistency over partition tolerance, guarding from 'split-brain' conflicts.

### 2.2.3 kube-scheduler

**What it is:** The control plane process responsible for deciding which worker node each newly created Pod should run on, kind of like a shceduler for an os, but without the preemption.

- **Decoupled Decision Maker:** The scheduler never actually starts a container or pulls an image itself; that responsibility belongs entirely to the kubelet running on the chosen node, which is responsible for 'touching' the pods. It it not able to allocate a position to a node that has just entered the cluster, if we want dynamicity we should use an 'extension' of kubernetes like the Karpenter or the 'cluster-autoscaler'.
- **Two-Phase Placement Engine:** Placement happens in two distinct passes over the available nodes.
  - **Filtering (Predicates):** The scheduler first eliminates any node that simply cannot host the Pod, for reasons such as insufficient CPU or memory, an incompatible taint, or a missing required node label.
  - **Scoring (Priorities):** Among the nodes that remain, it then ranks each one by fit — for example how evenly workloads are spread across zones, proximity to needed data, or overall resource efficiency — and assigns the Pod to whichever node scores highest. Mainly used for reallocating pods after failures or on scaling.

### 2.2.4 Cluster Autoscaler / Karpenter

**What it is:** A component that runs *alongside* the control plane, rather than as one of its core pieces, whose job is to add or remove worker nodes based on demand, not part of the default installation of kubernetes.

- **The Trigger:** Both tools watch the API server for Pods stuck in the `Pending` state because no current node can satisfy their resource requests, and treat that backlog as a signal that more capacity is needed.
- **Cluster Autoscaler:** The older, more established of the two. It works in terms of pre-defined node groups (an AWS Auto Scaling Group, a GCP Managed Instance Group, etc.) and simply scales those groups up or down, meaning new nodes come in whatever instance size and shape the group was already configured with.
- **Karpenter:** A newer, more flexible alternative that provisions individual nodes directly against the cloud provider's API, without needing pre-defined node groups. It picks the specific instance type, size, and even spot-vs-on-demand mix that best fits the exact Pods waiting to be scheduled, which typically means faster and more tightly bin-packed provisioning.
- **Scaling Down:** Both tools also work in reverse, continuously looking for nodes that are underutilized or empty and safely draining and terminating them to avoid paying for idle capacity.
- **Division of Labor:** Once a new node registers with the API server and is marked `Ready` (via the Node Controller in `cloud-controller-manager`), it becomes just another candidate node, and it is `kube-scheduler` — not the autoscaler — that decides which of the pending Pods actually land on it.

Finally it should be noted that the constituents of a cluster are in and of themselves deployed via kubernetes, kubernetes DNS provides naming and discovery for the services defined, also a replicated service, additionally there is the kubernetes proxy that's responsible for load balancing and traffic routing over the network of the cluster.

## 2.3 Kubernetes: the worker node

The most important moving parts are the data plane nodes. Every worker node runs the same three-piece stack: an agent, a runtime, and a network proxy.

```
Worker Node
|-- kubelet
|-- Container Runtime
|-- kube-proxy
```

### 2.3.1 kubelet

**What it is:** The main Kubernetes agent running on every worker node.

- Communicates with the `kube-apiserver` to learn what should be running on its node.
- Receives Pod specifications describing the containers, images, and resources a Pod needs.
- Ensures the assigned Pods are running by starting their containers via the container runtime.
- Monitors containers to confirm they stay in the expected state.
- Reports node and Pod status back to the control plane so it always has an accurate picture.
- Performs health checks and restarts containers when necessary, enabling self-healing.

### 2.3.2 Container Runtime

**What it is:** The container runtime is responsible for actually running the containers. Examples include:

- `containerd` – the most widely used runtime, and the default in most managed Kubernetes offerings.
- `CRI-O` – a lightweight runtime built specifically for Kubernetes.

The kubelet communicates with the runtime through the Container Runtime Interface (CRI), so any compliant runtime can be swapped in without changing kubelet itself.

### 2.3.3 kube-proxy

**What it is:** `kube-proxy` provides the networking functionality required for Kubernetes Services.

- Maintains network rules on the node, typically using `iptables` or `IPVS`.
- Enables communication with Services by resolving their stable virtual IP to real Pod addresses.
- Routes traffic to the appropriate Pods, load-balancing across all healthy backends.
- Implements Service types such as `ClusterIP` and `NodePort`, giving each a working network behavior.

## 2.4 Kubernetes: the object kinds

Every kind of Kubernetes object shares the same `apiVersion`/`kind`/`metadata`/`spec`/`status` shape and the same declarative contract — you write `spec`, Kubernetes writes `status`, and a controller continuously reconciles the two. What differs from one kind to the next is what that controller actually *does*: a Pod just runs containers, a Deployment manages rolling updates, a Job runs something to completion. The rest of this section is a reference card for each of the kinds you'll use most, showing how they relate to one another, what operations are available on each, and a minimal YAML example.

### 2.4.1 How the Kinds Relate

- **Deployment** owns a **ReplicaSet**, which owns **Pods**.
- **CronJob** owns a **Job**, which owns **Pods**.
- **DaemonSet** owns **Pods** (one per node).
- **StatefulSet** owns **Pods** (stable identity and storage).
- **Service** selects **Pods** — it does not own them.
- **ConfigMap** and **Secret** are mounted into **Pods** — again, no ownership.

**Solid arrow** = owns / creates (garbage-collected together via `ownerReferences`). **Dashed arrow** = selects or mounts into (no ownership, no cascading delete).

### 2.4.2 Pod

**What it is:** The smallest deployable unit in Kubernetes — one or more containers that share a network namespace and storage volumes, and are always scheduled together on the same node. Pods are described in a manifest, that describes them to kubernetes sincethey are the core object, pods can consist of multiple containers or can host one and communicate over the network, each one is controlled by a cgroup for resource limits and will share the appropriate namespaces with neighboring , not identical pods. Most important design decision for splitting the application's components into pods is what scaling needs are there for each, since a pod is the equivalent of an atom for kubernetes. Kubectl will accept and store teh manifests in etcd, then the scheduler will take over, and will ensure if possible to distribute the pods onto different machines for reliability.

**You can:**

- **Create** — `kubectl apply -f pod.yaml` → `POST /api/v1/namespaces/{ns}/pods`.
- **Inspect** — `kubectl get/describe pod` → `GET` the Pod resource or its `status` subresource.
- **Exec into it** — `kubectl exec` → `POST .../pods/{name}/exec`.
- **Read logs** — `kubectl logs` → `GET .../pods/{name}/log`.
- **Port-forward** — `kubectl port-forward` → `POST .../pods/{name}/portforward`.
- **Kill it** — `kubectl delete pod` → `DELETE .../pods/{name}`. If it's owned by a ReplicaSet, DaemonSet, or StatefulSet, a replacement appears automatically.
- **Cannot** scale, roll back, or migrate itself — Pods are disposable by design; those behaviors belong to the controllers above it. Kubectl exposes a basic api that's similar to docker for inspecting pods and scheduling them, that's because they can be almost identical in content.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

### 2.4.3 ReplicaSet

**What it is:** A controller that guarantees a fixed number of identical Pods are running at all times, matching Pods purely by label selector rather than by name. These provide redundancy, scaling and sharding, and forms the basis for the reliabilit yand fault tolerance of kubernetes. It's designed to group pods together and expose them as one, management takes place via reconciliation loops. This is the first instance of the configuration of the desired state defined earlier. What's being stated is the number of pods and their configurations, mechanisms have been defined to 'reconcile' each and every discrepancy from the blueprint. Labels are used to mount a pod to a replica set, also replicas are not coupled with the services mentioned later in this thesis about service discovery, dns and load balancing because it goes against the loose coupling principles of kubernetes.

**You can:**

- **Create** — `kubectl apply -f rs.yaml` → `POST .../replicasets`.

- **Scale** — `kubectl scale rs -replicas=N` → `PATCH .../replicasets/{name}/scale`.
- **Delete** — `kubectl delete rs` → cascades to its owned Pods via `ownerReferences`.
- **In practice** you rarely create these directly — a Deployment creates and manages them for you.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

### 2.4.4 Deployment

**What it is:** The standard controller for stateless applications, managing ReplicaSets on your behalf so you get rolling updates, rollbacks, and self-healing without touching Pods directly.

**You can:**

- **Run / create** — `kubectl create deployment` or `apply -f deploy.yaml` → `POST .../deployments`.
- **Scale** — `kubectl scale deployment -replicas=N` → `PATCH .../deployments/{name}/scale`.
- **Update (rolling deploy)** — `kubectl set image deployment/x c=img:v2` → `PATCH spec.template`; a new ReplicaSet is created and scaled up while the old one scales down.
- **Roll back** — `kubectl rollout undo deployment/x` → reverts to a previous ReplicaSet revision.
- **Pause / resume a rollout** — `kubectl rollout pause | resume`.
- **Restart** — `kubectl rollout restart` → patches a pod-template annotation, forcing fresh Pods even with no spec change.
- **Stop / kill** — `kubectl delete deployment` → `DELETE ...`; cascades to its ReplicaSets and their Pods.
- **"Migrate"** — Kubernetes never live-migrates a running Pod. Moving work off a node means cordoning it (`kubectl cordon`) and draining it (`kubectl drain`), which evicts its Pods so the Deployment's controller reschedules replacements elsewhere.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
```

```
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

### 2.4.5 StatefulSet

**What it is:** Like a Deployment, but for workloads that need a stable, unique network identity and stable per-Pod storage that survives rescheduling — think databases and other clustered stateful apps.

**You can:**

- **Create / scale / delete** — same commands as a Deployment, but Pods are created, scaled, and terminated in order (`web-0`, `web-1`, `web-2`, ..., reverse order on scale-down).
- **Update** — `spec.updateStrategy` is either `RollingUpdate` (default, ordered) or `OnDelete` (you delete Pods manually to trigger their replacement).
- **Stable identity** — each Pod keeps the same hostname and, via `volumeClaimTemplates`, the same `PersistentVolumeClaim` even after being rescheduled to a different node.
- **Delete** — `kubectl delete statefulset` removes the Pods but, by default, leaves the PVCs (and their data) behind.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "web"
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
  volumeClaimTemplates:
    - metadata:
        name: www
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

### 2.4.6 DaemonSet

**What it is:** A controller that ensures exactly one copy of a Pod runs on every node (or every node matching a selector) — the pattern used for log collectors, node monitoring agents, and network plugins. Typically these are the equivalent of the daemon, background running tasks

that are present in oses, and accordingly they have monitoring roles, and this is achieved by replicating one node to each node, installing in this way a daemon.

**You can:**

- **Create** — `kubectl apply -f ds.yaml` → `POST .../daemonsets`.
- **"Scale"** — not by replica count; the Pod count tracks the number of matching nodes automatically, so it grows or shrinks as nodes join or leave.
- **Update** — `RollingUpdate` (default) or `OnDelete`, same idea as StatefulSet.
- **Delete** — `kubectl delete daemonset` removes the Pod from every node.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      containers:
        - name: node-exporter
          image: prom/node-exporter
```

Hightower states: "ReplicaSets need to be run when the application is completely decoupled from the node you are using, while DaemonSets are for architectures where pods with your whole application must be present on all nodes of the cluster". There is however a way to specify a node subset for specific deployments, labels can be set as selectors for those cases.

### 2.4.7 Job

**What it is:** A controller for run-to-completion workloads — it tracks successful completions rather than keeping Pods alive forever, and retries failed Pods up to a limit. A job is the creation of a pod that runs until termination, batch jobs, database migrations and things that need to happen once and completely are the scope.

**You can:**

- **Run** — `kubectl create job` or `apply -f job.yaml` → `POST .../jobs`.
- **Control throughput** — `spec.completions` and `spec.parallelism` decide how many Pods must succeed and how many can run at once.
- **Suspend / resume** — `kubectl patch job x -p '{"spec":{"suspend":true}}'` pauses it without deleting it.
- **Stop / kill** — `kubectl delete job -cascade=foreground|orphan` controls whether its Pods are deleted along with it.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
spec:
  completions: 1
  parallelism: 1
```

```
  template:
    spec:
      containers:
        - name: migrate
          image: myapp/migrate
      restartPolicy: Never
```

Once again jobs are managing and deploying pods to get the job done, and in so doing we can achieve parallelization of workloads and even define work queues.

### 2.4.8 CronJob

**What it is:** A scheduler that creates a new Job on a cron-style schedule — the Kubernetes equivalent of a crontab entry, but producing real, trackable Job/Pod objects each run.

**You can:**

- **Create** — `kubectl create cronjob` or `apply -f cronjob.yaml` → `POST .../cronjobs`.
- **Suspend** — `kubectl patch cronjob x -p '{"spec":{"suspend":true}}'` stops new runs without deleting the schedule.
- **Control overlap** — `spec.concurrencyPolicy`: `Allow`, `Forbid`, or `Replace`.
- **Trigger manually** — `kubectl create job -from=cronjob/x manual-run-1` runs it immediately, outside the schedule.
- **Delete** — `kubectl delete cronjob` removes the schedule and its recent Jobs.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: backup
              image: myapp/backup
          restartPolicy: OnFailure
```

### 2.4.9 Service

**What it is:** A stable virtual address that load-balances traffic to whichever Pods currently match its label selector, so clients never need to track individual Pod IPs.

**You can:**

- **Create** — `kubectl expose deployment/x -port=80` or `apply -f svc.yaml` → `POST .../services`.
- **Change routing** — editing `spec.selector` or `spec.ports` re-routes live traffic as soon as `kube-proxy` picks up the updated Endpoints, no Pod restart needed.
- **Choose exposure** — `spec.type`: `ClusterIP` (internal only), `NodePort` (also exposed on every node), or `LoadBalancer` (provisions a cloud load balancer via `cloud-controller-manager`).
- **Delete** — `kubectl delete service` removes the virtual IP/load balancer; it never touches the backing Pods themselves.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Generally, service discovery is a contiguous problem, and abstracts/solves the discovery -literally- of the actual pod that's used to implement the actual service, even a routing table can be used, but in this case specifically we care about dynamicity of this since there are multiple levels of the stack from os to hardware to pod and to container that may be changing. This is why traditional DNS fails, SRV records are an alternative, however going further more suitable to the specific domain and application routing needs to be set up, perhaps with specialized or custom software. For outwards port forwarding there is the nodeport feature that's designed for allowing traffic in one namespace, by being able to reach just one node of the cluster.

### 2.4.10 ConfigMap & Secret

**What it is:** Two nearly identical objects for injecting configuration into Pods without baking it into the container image — a ConfigMap for plain values, a Secret for sensitive ones (base64-encoded at rest, and typically further protected by encryption-at-rest and RBAC).

**You can:**

- **Create** — `kubectl create configmap/secret` or `apply -f` → `POST .../configmaps` or `.../secrets`.
- **Consume** — mount as a volume (files appear under a directory) or inject as environment variables via `envFrom`/`valueFrom`.
- **Update** — `kubectl edit` or `apply` → `PATCH ...`; volume-mounted files update on the kubelet's next sync automatically, but env-var injections require the Pod to restart to pick up the change.
- **Delete** — `kubectl delete configmap/secret`; Pods already using the old data keep running until they're restarted.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
---
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcmQ=   # base64-encoded
```

### 2.4.11 The object model, and where each kind fits the engine

This is what the context of Kubernetes encompasses: below are the nouns and verbs of Kubernetes, making it clear what actions and objects we can handle for the management of the

lifecycle of our userspace programs.

```
Pod             one+ co-scheduled containers sharing net+IPC ns (localhost, sidecars)
Deployment      N interchangeable STATELESS pods, rolling updates   -> WORKERS
StatefulSet     stable names/storage, ordered ops                  -> COORDINATOR quorum
Job / CronJob   run-to-completion / scheduled batch                -> occasional batch only
DaemonSet       one pod per node                                    -> node agents
Service         stable virtual IP + L4 LB over a pod set
Headless Svc    one DNS A-record per pod (no VIP)                   -> peer discovery
ConfigMap/Secret  configuration / credentials
PVC / PV / StorageClass  durable volumes via CSI                    -> logs, DB state
Ingress / Gateway  north-south HTTP entry                          -> Submit front door
HPA / VPA       autoscale replicas / right-size requests           -> scale workers
PodDisruptionBudget  cap voluntary disruption                      -> protect quorum
NetworkPolicy / RBAC / SA / securityContext  least privilege
CRD + controller   extend the API with your own object + reconcile loop
```

> **Buy consensus; do not autoscale it.** A quorum's size is a correctness parameter, not a capacity knob — an HPA must never resize the coordinator StatefulSet, because adding a member is a membership-change protocol (Chapter 3), not a replica bump. And run *your own* etcd: sharing the cluster's control-plane etcd couples your load to the API server's health, a documented way to break a cluster.

Durability on Kubernetes has its own caveats: a PVC is not a backup (snapshot to object storage and *test restore*); the log's device must give low-latency `fsync`, so prefer local NVMe over network storage whose sync is a network round trip.

## 2.5 API gateways, ingress, and service mesh

Traffic splits into *north-south* (clients → cluster) and *east-west* (service → service), and each has an edge.

**North-south** is the Ingress / Gateway API and its controller (Envoy, NGINX, Traefik): one entry point that terminates TLS, routes by host/path, enforces authentication and rate limits, and splits traffic for canary or blue-green rollouts. For the engine this is the *Submit* front door: it authenticates callers, applies per-tenant rate limits and load shedding (Chapter 3), and routes to the coordinator leader — so the admission and backpressure logic lives at a single, observable choke point rather than being scattered. The engine's heads, though, speak their own protocol over plain TCP, not HTTP, so the gateway in front of them has to work one layer down, as a *stream proxy*: Apache APISIX, for example, can proxy raw TCP and UDP connections — as it does for databases, DNS or syslog — alongside its HTTP routes [9], forwarding each connection to a head without looking inside it.

**East-west** is optionally a *service mesh*: a sidecar proxy (Envoy) injected beside every pod that transparently adds mutual TLS (service identity), retries, timeouts, circuit breaking, and per-call telemetry. This is exactly the fault-tolerance and RPC-hardening machinery of Chapter 3, *externalized* out of the application into the platform. The trade is real overhead (a proxy hop and CPU per pod) and operational complexity, so a mesh earns its place at scale or across trust boundaries, not by default; a C engine that already implements deadlines, retries with budgets, and mTLS in its own RPC layer may reasonably skip it and keep the control it already has.

## 2.6 The engine on Kubernetes: which entities it uses, and for what

The sections above describe the entities in general. This one maps them onto the engine as it is actually deployed (`engineCJavaKubernetes`, `deploy/k8s/`): two ordinary PCs joined by a

1 Gb cable and running k3s, one image (`engine:dev`) for every role, and the engine of Chapter 3 otherwise unchanged. Kubernetes contributes placement, restarts, stable names and the number of nodes; it does not take over any decision the engine already makes.

| entity | object | role in the engine |
|---|---|---|
| Namespace | `engine` | everything below; deleting it removes the deployment |
| Pod | `engine-K` | one engine node: a C worker and a Java head |
| StatefulSet | `engine` | the nodes, with stable names |
| StatefulSet | `etcd` | the cluster layer's store (Section 3.10) |
| Service | `engine` (headless), `etcd` | one DNS name per node; one for the store |
| Job | `submit` | one batch of work, then it exits |
| ConfigMap | `engine-config` | threads, slots, what to submit |
| ScaledObject | `engine` (KEDA) | the number of engine pods, from worker CPU |

**The current deployment.** The engine as it stands now (Section 3.11) is deployed by a separate project, `engineKubernetes`, built from the two projects beside it: `engineJava` for the engine and `pipeIn` for the work. It keeps the central decision of the earlier deployment — one pod is one engine node, a worker and a head sharing a directory, resolving artifacts and running jobs exactly as they do on a machine — and changes what surrounds it:

- two machines on an ordinary LAN instead of a dedicated cable;
- exactly two engine pods, one per machine, instead of an autoscaled number;
- no submit Job: work is pushed to the heads, and results fetched from them, over the network, by pipeIn speaking the engine's own protocol;
- two NodePort Services, so that any machine on the LAN can do that;
- placement between the two nodes by the heads themselves: work is sent to `engine-0`, and its head, started with `--spread`, ships every other ready job to `engine-1`.

| entity | object | role in the current deployment |
|---|---|---|
| StatefulSet | `engine`, 2 replicas | one node per machine (required anti-affinity) |
| StatefulSet | `etcd` | unchanged: the heads' live claims |
| Service | `engine` (headless) | one DNS name per node, which each head advertises |
| Service | `engine-0-lan`, `engine-1-lan` | NodePorts 30100 and 30101: each head, reachable from the LAN |
| ConfigMap | `engine-config` | etcd's address, worker threads, advertised slots |

It is built and checked — the manifests parse and agree with each other on every shared path, variable, port and selector, and the image is built from sources alone — and it comes with its own test loads: a script that follows both heads' logs as jobs are received, shipped, run and asked for, a suite that runs every load on freshly restarted pods and verifies every result, and the timed comparison with bare nodes of Section 3.11.1. At the time of writing it has not yet been run on the two machines, and no result is claimed for it.

### 2.6.1 Pod: one pod is one engine node

The head and the worker talk over a unix socket and reply through the filesystem (Section 3.9), so they must share a machine and a directory. That is exactly what a Pod guarantees: two containers — the worker (`/opt/engine/node`) and the head (`JobServerCli` on port 9100) — scheduled together, sharing an `emptyDir` volume mounted at `/data` that holds `engine.sock`, `functions/` and `datasets/`. Each container asks for CPU (worker 1, head 0.1): the scheduler packs pods by these requests and the autoscaler measures against them. The pod is Ready when the worker's socket exists and the head is listening. The `emptyDir` makes the lifetime explicit: a node's results live exactly as long as its pod.

**In the current deployment** the pod is the same pair, with two changes that follow from the engine itself. The worker now serves exactly one head for its whole life and removes its socket once that head connects, so "the socket exists" is true only *before* the node is working and can no longer be a readiness test; the pod is Ready when the head is listening, which it does only after it has reached its worker. And because both containers start at once, the head retries the worker's socket for up to thirty seconds rather than failing on the first attempt; if it later loses the worker it exits, and the kubelet restarts whichever container stopped — the worker, too, exits when its one head goes. The node therefore restarts as a pair without any ordering in the manifest.

The organisation of one engine pod, as `k8s/engine.yaml` declares it:

```
pod engine-K   (StatefulSet engine, one per machine)
+-------------------------------------------------------------------------+
|  container worker  /opt/engine/node         container head  java ...     |
|    --socket /data/engine.sock                 JobServerCli --port 9100    |
|    --datasets /data/datasets                  --node /data/engine.sock    |
|    --threads $(ENGINE_THREADS)                --etcd $(ETCD_ENDPOINTS)    |
|    workingDir /data (-> ./functions)          --node-id $(POD_NAME)       |
|    requests: cpu 1, 256Mi                     --advertise $(POD_NAME)     |
|    no probe                                     .engine.$(POD_NAMESPACE)  |
|                                                 .svc.cluster.local:9100   |
|                                               --slots $(ENGINE_SLOTS)     |
|                                               --spread                    |
|                                               requests: cpu 100m, 256Mi   |
|                                               readiness: tcp 9100         |
|          \                                       /                      |
|           +---- emptyDir /data: engine.sock, functions/, datasets/ ---+   |
|  shared by both: the network namespace (localhost), the IPC namespace     |
+-------------------------------------------------------------------------+
      ^ :9100 via engine-K.engine (headless)      ^ NodePort 3010K (engine-K-lan)
      | the other head                            | pipeIn on the LAN
```

Everything the two containers exchange goes through the volume: the socket that carries the objects to the worker, the `.so` files both of them may write, and the results the worker writes and the head reads. Nothing is exchanged through the network namespace they also share, and nothing through their common IPC namespace — which is what the controller of Chapter 4 would use instead.

### 2.6.2 StatefulSet rather than Deployment

A Deployment's pods are interchangeable and renamed on every restart. An engine node is not interchangeable: it advertises an address in etcd that other nodes dial. A StatefulSet gives every pod a stable ordinal name (`engine-0`, `engine-1`, ...) that survives restarts, which is why it is used for the nodes even though they hold no durable state. Two properties follow and both matter. Pods are created in parallel (`podManagementPolicy: Parallel`), so scaling up is not serialised; and scaling down always removes the highest-numbered pods, busy or not — taking the jobs queued on them and the results in their `emptyDir` with them. A topology spread constraint spreads the pods evenly over the two PCs.

**In the current deployment** the StatefulSet has exactly two replicas, and a *required* anti-affinity on the host name replaces the spread constraint: two engine pods may never share a machine. With both machines up, `engine-0` and `engine-1` land one on each; with one, the second pod stays Pending — a visible signal rather than two nodes quietly doubled up on one machine, where they would read the same load and hide the point of the exercise.

The store runs as a second, separate StatefulSet (`etcd`), one member on an `emptyDir`. It is the engine's own etcd, not the cluster's control-plane store, as recommended above; losing its

data costs nothing, because every key is a leased live claim that the heads republish within a second.

### 2.6.3 Service: headless, for naming only

The `engine` Service is headless (`clusterIP: None`): it provides one DNS record per pod, `engine-K.engine.engine.svc.cluster.local:9100`, and no load balancing. Each head advertises that name in etcd, and the submitter dials it. This is deliberate: which node receives a job is the engine's decision (round robin over the nodes etcd lists as live), so a Service that picked a pod on its own would contradict the placement rule. The `etcd` Service, by contrast, is an ordinary one — a single stable address for the store.

**In the current deployment** the headless Service is used between the heads: each advertises `engine-K.engine.engine.svc.cluster.local:9100` in etcd, and a head that needs a `.so` or a dataset looks the other one up there and dials it directly; the same link carries the jobs one head ships to the other. Two more Services open the heads to the LAN, one *per pod*: `engine-0-lan` selects the pod named `engine-0` (by the `statefulset.kubernetes.io/pod-name` label the StatefulSet sets) and exposes it on port 30100 of every machine, `engine-1-lan` does the same on 30101. They are NodePorts rather than one balanced Service because a client must be able to reach a *particular* head: a result lives on the node that produced it, and fetching it means asking that node — a balancer would answer `NOT_FOUND` for half the results at random. The engine has no authentication, so these ports open "run this code" to the whole LAN — acceptable on a home network, not on a shared one.

### 2.6.4 Job: a batch, submitted and gone

A submission is run-to-completion work, so it is a Job. Its pod runs three things in order: an init container that builds the batch (`jobs.bin` plus the function and the datasets it names); a worker as a *native sidecar* (an init container with `restartPolicy: Always`, Kubernetes 1.29+), because the head refuses to start without a local worker; and the head in file mode, `JobFileCli --spread --slots 0`. With zero slots the submitter runs none of the jobs itself and ships every one, with its function and data, to the engine pods — necessary, because a Job's pod and its disk disappear when it completes, and anything computed there would be lost.

**In the current deployment** there is no submit Job. A head listens on TCP and speaks the engine's protocol to anyone, so pipeIn, from any machine on the LAN, pushes the job file to the heads (`send`) and asks them for the results by job identifier (`fetch`): a `REQUEST` for the DATA named by the job, answered with the DATA or with `NOT_FOUND`, repeated until every result is in and then compared byte for byte with the answers pipeIn computes itself. Jobs are sent to `engine-0` alone, and its head spreads them: each job that becomes ready runs on its own worker or is shipped, with its `.so` and data, to `engine-1`, in turn. A shipped job is marked `SHIPPED` in its flags — which are not part of its identity, so its result keeps its name — and the receiving head runs it rather than passing it on. The dependent chains therefore cross machines: a head whose job depends on a result produced on the other node asks that node's head for it once; that head answers that it will send the result, and pushes it the moment its worker has written it (Section 3.10). A head addresses such requests only to another head — given on the command line or listed in etcd — and never to a client that connected to push work. The functions are compiled inside the image, by the compiler whose output the workers will load, and the job file is taken from the image; a job built on the submitting machine would name different bytes, and a different identifier.

### 2.6.5 ConfigMap and the downward API

`engine-config` holds the tunables — etcd's address, worker threads, advertised slots, and what the submit Job generates. Containers receive it as environment variables, and Kubernetes substitutes them into the command line (`--threads $(ENGINE_THREADS)`). A pod's own name, namespace and IP come from the downward API (`fieldRef`), which is how a head knows its node identifier and the address to advertise without any configuration per pod. A changed ConfigMap reaches running pods only on a restart, which the deployment script performs.

In the current deployment the ConfigMap holds three values — `ETCD_ENDPOINTS`, `ENGINE_THREADS` (4) and `ENGINE_SLOTS` (16) — and the head builds both its node identifier and its advertised address from `POD_NAME` and `POD_NAMESPACE`, taken from the downward API: `engine-0` advertises `engine-0.engine.engine.svc.cluster.local:9100`.

### 2.6.6 Autoscaling: KEDA, the HPA and metrics-server

KEDA is set up in three steps, all driven by `deploy/k8s.sh keda`. metrics-server comes first, because the CPU trigger is an HPA underneath and the HPA reads pod CPU from it; k3s ships it, and on other clusters the script installs it. KEDA itself is then installed into its own `keda` namespace from its Helm chart (or its release manifest, when Helm is absent), and the script waits until all three of its components are available — the operator, its metrics API server, and its admission webhook, which refuses any ScaledObject until it answers. Last comes the ScaledObject, `deploy/k8s/autoscale/engine-scaledobject.yaml`, kept outside the main manifests because its kind exists only once KEDA's CRDs do. It names the `engine` StatefulSet as its target, keeps between 2 and a configurable maximum of pods (`KEDA_MAX`, about the two machines' cores together), triggers on the `worker` container's CPU at a 70 % target of its 1-CPU request, and shapes the HPA's behaviour: at most two new pods every 30 s on the way up, and on the way down ten quiet minutes, then one pod every two minutes. `deploy/k8s.sh keda-off` deletes it and returns the StatefulSet to a fixed size.

The number of engine pods is managed by a KEDA ScaledObject, which KEDA turns into a Horizontal Pod Autoscaler. Its trigger is the CPU of the *worker* container only — the head idles — read through metrics-server, which k3s ships. The HPA computes desired $= \lceil n \cdot u/\text{target} \rceil$ for $n$ pods of average utilisation $u$. With one worker thread per pod, a busy pod reads at most 100 %, so one step can grow the cluster by at most 100/target: at a 70 % target the cluster crept from 2 to 3 pods and stayed there, because newly started pods idle until the next batch and pull the average below the target; at 25 % it grew from 2 to 8 pods in about 70 s. In keeping with the rule above, only the engine StatefulSet is autoscaled — never `etcd`.

Scaling interacts with the push model. A job is bound to the pod it was sent to, so a new pod receives work only from the next batch, and scale-down would remove pods that still hold queued jobs. The stress test therefore disables scale-down until every job has a result, and the deployment script pauses the ScaledObject while it restarts the pods, so the two controllers do not fight over the replica count.

**In the current deployment** autoscaling is not used: the aim is two machines, one node each, and a fixed pair is what makes cross-machine behaviour observable. Nothing in the engine prevents it — a new pod publishes itself to etcd and becomes a head the others can reach — but under the push model it would receive work only from the next submission, as described above.

### 2.6.7 Scheduler, kubelet and runtime, and the network

These are used as they come. The scheduler filters nodes by the CPU the pods request: at 1.1 CPU per engine pod, the two PCs hold 35 of them (14 and 21). Each kubelet starts containers through k3s's containerd, which pulls `engine:dev` from an image registry on the first PC. Pod

traffic crosses between the machines as flannel VXLAN, pinned to the cable (`--flannel-iface`), so a job shipped to a pod on the other PC travels over the direct link.

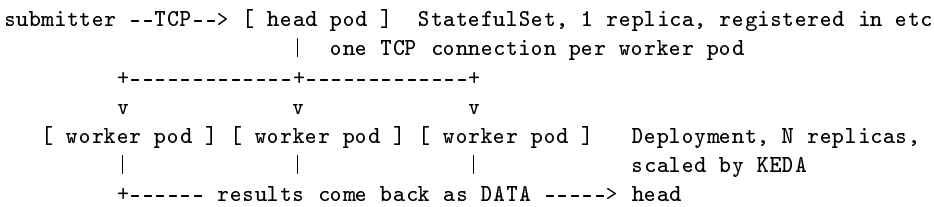
**In the current deployment** the machines share an ordinary LAN, and flannel is pinned to each machine's LAN interface instead. Two scripts set the cluster up: one on the first machine installs the k3s server and an image registry and opens the ports the machines need between them (6443 for the API, 8472/udp for flannel, 10250 for the kubelet) and those the LAN needs (30100 and 30101); one on the second machine joins it as an agent that pulls the engine image from the first. Both engine pods therefore run the same image, built once.

### 2.6.8 What is deliberately not used

- **Deployment, ReplicaSet** — nodes need stable names; see above.
- **DaemonSet** — one node per machine would forbid scaling by pods.
- **PersistentVolumeClaim** — results are on `emptyDir`; a claim per pod (`volumeClaimTemplates`) is the next step if results must outlive scale-down.
- **Ingress, Secret, CronJob** — no external clients, no credentials, no schedule yet.
- **Cluster Autoscaler, Karpenter** — the machines are two fixed PCs; KEDA scales pods within them, and a Pending pod is the signal that they are full.

### 2.6.9 An alternative split: one head pod, many worker pods

The deployment above keeps the node of Section 3.9 intact: every pod is a head and a worker, and the engine scales by whole nodes. The other natural split puts a single head in one pod and the workers in many pods of their own — a coordinator with executors instead of a set of peers.




**What has to change in the engine.** The present node relies on two things that only hold inside one pod. The worker listens on a unix socket, so it would need to accept TCP as well; the framing needs no change, since the same objects are valid on any byte stream (Section 3.3.1), and the head can already write to a `host:port` — it does so when it ships work to a peer. The harder change is the reply path: a worker that shares no directory with the head cannot answer by writing a file. The natural answer within the object model is to send the result back on the same connection as an ordinary DATA object, which the head stores and publishes as it does today. That gives the protocol its first reply, and with it something the engine currently lacks: the head knows which jobs each worker still holds.

**How it would be deployed.**

- **Workers: a Deployment**, not a StatefulSet. A worker no longer advertises an address that anyone else dials by name — only the head talks to it — so its identity does not matter and interchangeable replicas are the right model. Each pod is the C worker alone, one thread, its function cache and datasets on an `emptyDir`.
- **Discovery: a headless Service** over the worker pods. Its DNS name resolves to every ready worker's address, and the head re-resolves it periodically to pick up pods that KEDA has added. An ordinary Service would be wrong here: it balances *connections*, not jobs, and the head holds one long-lived connection per worker, so placement must stay the head's decision.

- **Head: a StatefulSet of one replica** with a stable name, reached by submitters through an ordinary Service and registered in etcd as the single node. One head is also one owner of every job, which keeps the single-writer argument of Section 3.6 trivially true. Its catalogue wants a PersistentVolumeClaim, since it now holds every input and every result.
- **Autoscaling: KEDA on the worker Deployment only**, with the same CPU trigger. A new worker receives jobs as soon as the head opens a connection to it — not from the next batch, as in the present design — because the head, not a submitter, decides where each job goes.
- **Scale-down** still removes pods that may hold queued jobs, but no longer silently: the head sees the connection close and re-sends that worker's unfinished jobs elsewhere. A `preStop` hook and a termination grace period let a worker finish what it holds before it goes.

**What it costs.** Every job's data now crosses the network from the head to a worker, where today a node runs work on data it already holds, over a local socket — the head's link becomes the throughput limit, and with the two PCs, half the workers sit across the cable from it. The head is also a single point of failure for the whole cluster, where today losing a node loses only that node. What the split buys is workers that are stateless, interchangeable and cheap to add or remove; failures that are detected rather than silent; and one place that knows the state of every job. It is, in effect, the pull design's coordinator reached by a shorter step: keep the head's catalogue, give the worker a TCP listener and a reply, and let the head do the distribution a queue would otherwise do.

# Chapter 3

# The Engine

The engine is deliberately a *skeleton* of a complete distributed execution system. It implements job definition, identity, placement, dependency tracking, code shipping, and the feed of one job's output into another's input. It does not implement failure recovery, concurrent dispatch, or multi-head routing. Section 3.13 states each omission and where it would attach, because an honest account of what a prototype does not do is more useful than a diagram of what it might. While may be not production ready many of the limiting factors come into play and opportuniteis for proper analysis come up.

## 3.1 The high level design

The objective is a system that uses two or more computers to perform one task, or many, faster than any one of them could — and not in an embarrassingly parallel or otherwise trivial way. The design chosen is loosely coupled: the components still communicate constantly, but communication is built into the system in a way that scales and can be configured. Kubernetes' management of lifecycles, for example, can provide the basis for adding nodes to the network and starting workers on them.

Because the aim is not merely to fan work out, or to map and reduce it, but to compute results that depend on one another in non-trivial ways, the system accepts a set of *dependent* jobs. What is submitted is a DAG — a directed acyclic graph — whose edges say which jobs need the output of others before they can start; one or more jobs feed the graph's sinks. The workers' thread pools also make a simple fan-out possible: a job whose header marks it *parallelizable* can have its data divided into parts that run side by side.

## 3.2 The job model

The unit of work is the *job*, which begins its life as a pair: some data and a function. The function is a shared object (`.so`) loaded dynamically. In the intended deployment the shared objects are distributed to the nodes once and not sent again with every job, since most systems are built with the loads they will face in mind. Large jobs travel over TCP, because a lost piece of data would stall a whole pipeline and could leave its results unusable.

Each dataset and each function is named, and a job — one of each — is named too. The names are hashes: cheap to compute, identical wherever they are computed, and directly usable as keys for storing results, for instance in a distributed hash table. That matters most for a distributed system: if a result's location follows from its name, no node needs to know the whole graph or where each part of it runs — whoever needs a result takes the job's identifier and computes the node that holds it. It also means the worker needs no catalogue of functions and no prior knowledge of the work: everything it must know arrives in the bytes it has just read, and a short-lived job marked *no cache* leaves nothing behind.

The unit of data is usually a *dataset*: rows of fixed-width integer vectors, each row carrying its own identifier. The unit of code is a compiled shared object — or, more precisely, the identifier its hash gives it, which is the same everywhere and lets a leader distribute the code once the virtual machines, machines or containers of the cluster have started.

A *job graph* is a set of jobs together with their dependencies. A job's input is either a file or the concatenated rows of its dependencies' outputs; a job that consumes another's output names that job's identifier as its input. The graph is built and then walked in reverse, which establishes which jobs must run first.

A load balancer then decides, for each job, whether it runs on the local machine or is sent across the network. That balancer could be an L7 or L4 service of Kubernetes, or the heads themselves, at L4, over the connections they already hold. Where a result is *stored* is a separate choice: it could go to a machine chosen by the job's hash modulo the number of nodes, at the risk of saturating the network. A home internet connection may upload as little as 1 MB/s, so a job graph must compute a great deal per byte it moves to gain anything from this model of execution — which, in this DAG form, is a weakly synchronised pipeline.

```
worker 127.0.0.1:16001
worker 127.0.0.1:16002

job fibA  so=funcs/fib.so fn=fib data=input/vectors_a.bin flags=parallelizable
job fibB  so=funcs/fib.so fn=fib data=input/vectors_b.bin flags=parallelizable
job total so=funcs/sum.so fn=sum deps=fibA,fibB
```

Two producers compute Fibonacci numbers over independent inputs; a third consumes both. That last line is the whole point of the exercise — it is the edge across which one job's result becomes another job's input, on a different machine, without either job knowing the other exists.

Two programs implement this. The **head** reads the graph, assigns identities, decides placement, tracks dependencies, and dispatches. The **worker** receives one task, loads the function it carries, runs it over the data it carries, and replies. There is no third component: no coordinator, no metadata service, no queue.

## 3.3 One wire format

The experimental groundwork for this engine is one client/server pair, each playing a different role, and each of which grew its own frame layout. The engine's first design decision is therefore a negative one: there is exactly one definition of the format, and the head, the worker and every job function compile against it.

```
task frame     head -> worker
    [u32 totalLength][ 48-byte header | symbol | data | code ]

      0  magic  0xE9C1FEED     20  nameOffset    24  nameLength
      4  version               28  dataOffset    32  dataLength
      8  jobId  (u64)          36  codeOffset    40  codeLength
     16  flags                 44  reserved

reply frame    worker -> head
    [u32 status][u32 outLength][outLength bytes]

dataset
    [u32 magic][u32 version][u64 rowCount][u32 cols][u32 reserved]
    rowCount x [u64 rowId][int64 v0 .. v(cols-1)]
```

Offsets rather than a packed struct: the frame is self-describing, so the regions can be validated independently and a stale sender fails on magic or version rather than by silently misreading

a field. The reply carries a `status`, which is a small thing that matters — a failed job is *reported*, where the earlier prototypes signaled failure by dropping the connection and left the caller to infer the cause.

One class of bug in those prototypes is worth recording, because it appeared independently in every one of them. Each validated an attacker-controlled row count with arithmetic that overflowed before the check could reject it: a count near $2^{61}$ makes $8 + \mathit{count} \times 8$ wrap to a small value, the guard passes, a small buffer is allocated, and the loop then writes past it. In the datagram-based pairs this was reachable from a single unauthenticated packet. The engine's `dsBytes` bounds the count *before* multiplying, which is the only ordering that is safe, and the constant that bounds it (`DS_MAX_ROWS`) is part of the format rather than a local convention.

### 3.3.1 In `engineJava`: from task frames to objects

Albeit with some limitation he engine sends a *task frame*: header, symbol, data and compiled function in one message. Two things about it do not scale. A frame that carries its payload is never cheap — one run re-sent the same 15.5 KB function with each of 512 jobs, about 8 MB of redundant transfer. And a frame is not addressable: once several machines are involved, the useful questions are about names (who holds this dataset, send me that result), and "one task with everything inside" has nowhere to put them.

This is partly solved by moving around jobs as pairs of handles/identifiers of methods to run and data along with their metadata to run on.

Every object begins with the same 24-byte envelope:

| offset | size | field |
|---|---|---|
| 0 | 4 | magic |
| 4 | 2 | version |
| 6 | 2 | type |
| 8 | 8 | objectId |
| 16 | 8 | totalLength |

and there are four types:

- **DATA** — a 56-byte header and an opaque payload. A result is simply a DATA, so one job's output can be another's input with no conversion.
- **FUNCTION** — a 112-byte header and the compiled `.so`. Every function exports one entry point, `engineJob`.
- **JOB** — 96 bytes that carry nothing but identifiers: the function, the input and the output. Two flags, `DEPENDENT` and `SHIPPED`, sit outside the bytes the job's id is computed from.
- **REQUEST** — 56 bytes meaning "send me the object of this type with this id".

So moving jobs has become relatively cheap and also reading from files or pipes has become easier since we cutoff at exactly the same 96 bytes of payload. And because the id is at the same offset in every type, anything can be indexed or requested by name. There is no response type: the answer to a REQUEST is the object itself, or the REQUEST sent back marked `NOT_FOUND` — or `WAITING`, a promise to push the object once it exists, in a non blocking way.

## 3.4 Identity: ids computed from the bytes

Every identifier is a hash of the bytes it names: the first 64 bits of their SHA-256 digest, read big-endian, so that an identifier printed in hexadecimal is exactly the first sixteen characters of `sha256sum`:

- a **row id** is the hash of that row's value bytes, so a row keeps its identity no matter which job produced it or how many merges it has survived;
- a **job id** is the hash of the function name, then the code, then the input, so the same function over the same input is the same job — on any head, in any process, forever.

This is content addressing, and it is cheap to adopt at this scale while paying for itself several times over. It gives deduplication for free: two submissions of identical work collide by construction. It gives idempotence, because a repeated dispatch is recognizably the same dispatch. It gives a cache key for the function artifact, which is the mechanism by which the shared object stops traveling with every task. And it gives determinism that can be checked from outside the system: the row identifiers produced by the engine were reproduced by an independent implementation written against the format specification alone, which is a stronger statement about the format than any amount of internal testing.

The same keys would serve an external result store as well — a Redis instance, or a PostgreSQL stored procedure — since the form of every payload on the network is known in advance.

Inputs are content-addressed and job functions are pure, so any lost result can be recomputed by replaying its subgraph. The ingredients for lineage-based recovery — the model Spark uses in place of replicating intermediate data — are already present; only the bookkeeping is missing: the head that received a graph would have to keep it, and the set of results it has seen, so that it can tell which part to replay. This is noted again in Section 3.13.

## 3.5 DAG execution

We can complete more advanced and complex jobs by defining dependencies between jobs.

One job can take as data input the id of another job, it would then have to wait for that job to complete for its data to be available and for them to be resolved. Since two separate machines are being used right here, what we are implementing needs no coordination, we definitely know where the data is if they are not on our node. However this still warrants the existence of a sufficient mechanism for the above mentioned resolution. The simplest answer is to batch non blocking - asynchronous - communications, otherwise storms are possible to be observed and in worst case scenario each job would do a complete round trip before being executed properly.

Technically though a 'sea' of complex algorithms could be implemented, and in a large enough scale the total number of jobs would be executed in parallel to the extent possible for the system provided.

A simple starting policy is this: a producer inherits its consumer's node by default, which keeps the feed node-local; a producer marked `parallelizable` in the graph is spread to the next node instead. With the three-job graph above:

```
workers    fibA   fibB   total
   1        w1     w1     w1
   2        w2     w1     w1
   3        w2     w3     w1
   4        w2     w3     w1      (three jobs; w4 idle)
```

The flag is the only scheduling hint the graph carries, and it is enough to express the distinction that matters here: work that must be near its consumer, and work that should be spread away from it. What the rule removes is the case in which results are ready on one or two nodes and a third must find and collect them before it can start. That case could also be handled by name — knowing the job's hash, the node that needs the result asks for its bytes — on the assumption that the producer exists, will run, and will neither become a bottleneck on several fronts nor deadlock, not threads this time, but machines. The current engine takes exactly that second route: a dependent job names its producer, and a head that does not hold the result asks the other head for it once, and is sent it when it exists (Section 3.9) and with the current implementation it's a soft optimizatio nto have non blocking asyncronous requests.

## 3.6 Dependency tracking without consensus

Each job holds a counter of unsatisfied dependencies. When a job completes, the counter of every job depending on it is decremented; a job whose counter reaches zero is ready to dispatch. This is the standard dependency-counter formulation, and the standard treatment of it spends considerable effort on the race: two producers finishing simultaneously, both reading the same counter, both writing the decremented value, and the join silently never firing. The usual remedies are an atomic decrement in a transactional store, or a coordination service holding the counter on everyone's behalf.

The engine needs neither, and the reason is structural rather than clever. Placement is deterministic and the graph is known, so each job has exactly one owning head, and only that head ever writes that job's counter. A single writer cannot race with itself. The concurrency problem was not solved; it was *removed*, by an ownership rule that placement already had to establish for other reasons.

This property is worth stating precisely because it is the part that does not survive carelessly. It holds as long as ownership is unambiguous — which means it depends on every head computing the same placement, which in turn depends on every head seeing the same membership. Membership change, not failure, is the first thing that will break it, and it is the reason the design calls for a membership epoch stamped onto each graph at submission.

## 3.7 Shipping code

The worker receives the function as bytes and must run it without the artifact ever existing as a file. It creates an anonymous in-memory file with `memfd_create`, writes the received object into it, and calls `dlopen` on `/proc/self/fd/N`. Nothing touches disk, and because each task gets a fresh inode, the dynamic linker's path-based cache cannot return a previously loaded object that happened to share a name — a failure mode that is invisible until two jobs disagree about what a symbol means.

This mechanism was developed in the first of the four prototype pairs and is reused unchanged. The current engine gave it up: once code is named by its own hash, a file called `functions/<id>.so` is exactly as safe from the linker's path cache as a fresh inode, since two different objects can never share the name; it can be loaded once and cached by identifier, and any process on the node can find it. The prototypes established a ranking between the three ways a worker can obtain code, fastest first: a resident function compiled into the worker; flat position-independent machine code copied into a mapping made executable with `mprotect`; and a shared object loaded through the dynamic linker. The engine uses the slowest of the three deliberately. The ranking measures the *loading* step, which content addressing is about to make rare, and in exchange the shared object brings a symbol table, a defined ABI, and ordinary tooling — a flat blob has none of these, and its ABI in the freestanding prototype omitted the output capacity argument, which is precisely how it came to write past its buffer.

## 3.8 Deployment

The engine runs as a set of containers, one per node, on a user-defined network so that nodes address each other by name rather than by address. A single image serves every role — build environment, worker, and head — and the project directory is bind-mounted rather than copied in, so that rebuilding a binary does not mean rebuilding an image. Workers therefore all listen on the same port, distinguished by hostname.

The nodes are provisioned as debugging environments rather than as minimal runtime images: each carries `gdb`, `strace`, `tcpdump` and the traffic-control tools, and runs with the capabilities those require. This makes the properties discussed above directly observable — the frame header

can be read off the wire between two containers, the `memfd_create` and `dlopen` sequence can be traced as a task arrives, and artificial latency or packet loss can be introduced on a single node's interface to see how the engine behaves when the network stops being ideal.

Kubernetes is intentionally absent from this iteration. Everything Chapter 2 describes applies — a worker fleet is a Deployment, the head is a StatefulSet with stable per-pod DNS, and membership comes from the API server — but the engine's own mechanisms are easier to observe and to falsify when the orchestrator is not also in the picture. The step from named containers on a network to named pods on a cluster is small, and it is the natural next one.

`engineJava` takes that next step: it runs on Kubernetes as described in Section 2.6, one pod per node.

## 3.9 The `engineJava` node

A node is two processes on one machine: a C worker that runs jobs and a Java head that decides which jobs to run and where. They share only the wire definition and talk over a unix socket, in one direction.

### 3.9.1 The worker

The worker is about 1 200 lines of C. It accepts exactly one head, removes its socket so no one else can connect, serves that head until it leaves, then drains its work and exits. Inside, it has four parts:

- a **receiver** that reads DATA straight into memory, writes FUNCTION code to `functions/`, and queues JOBs;
- a **store**: one large `mmap(MAP_NORESERVE)` region with a bump allocator — 4 GB reserved, paid for only when touched;
- a **queue** in which jobs never move, only their state (WAITING, READY, RUNNING, COMPLETED);
- a **work-stealing pool**, one deque per thread. When it is full the receiver stops reading, which slows the head down through the socket instead of dropping work.

Each `.so` is opened once and cached by id. Dividing a job into parts was removed: no supplied function could use byte-boundary splits, so a job always runs whole.

### 3.9.2 The filesystem as the reply path

The worker never answers the head. It writes each result to a temporary name and renames it to `datasets/<jobId>.bin`; the file appearing *is* the reply, and the rename makes it atomic. That one write makes the result usable as another job's input, servable to another machine and publishable to the cluster. It is a convenience rather than a principle, though: it works only because both processes share a disk.

### 3.9.3 The head

The head is about 3 000 lines of Java in a strict ports-and-adapters layout. Its domain objects are views over bytes rather than parsed copies, and its copy of the wire constants is generated from the C header, so the two sides cannot drift apart. Jobs can come from a job file, from another node over TCP, or from the command line; all three feed the same resolver.

A job is *ready* when the head holds both its `.so` and its input. Otherwise it waits, and is rechecked whenever an object arrives and once a second, when the head also asks the other node for anything still missing. A ready job runs on the local worker, or — with `--spread` — alternately here and on the other nodes listed in etcd. A job sent elsewhere is marked `SHIPPED` so that it is never passed on again. Running 30 000 jobs exposed two races in this path (the

resolver reading the catalogue twice, and the catalogue rebuilding its index from a stale listing); both are fixed.

## 3.10 The cluster layer

Two machines need to know two things about each other: how busy each is, and what each holds. Both answers live in etcd, and nothing else does:

| key | value |
|---|---|
| `/engine/nodes/<nodeId>` | `cpu\|memory\|running\|slots\|address` |
| `/engine/data/<nodeId>/<dataId>` | empty — the key is the fact |

Jobs, code and results travel directly between nodes over TCP. etcd is a consensus store, where every write is a replicated round trip, so it carries only the small facts needed to make decisions. Each head holds a five-second lease, renewed every second, and attaches every key to it; when a node dies its keys simply expire. To keep writes low, a head publishes only what changed — at most 256 keys per tick — with a full rewrite every thirty ticks to repair any drift.

**Moving and fetching work.** A head ships a job either when it is busy and a peer is clearly lighter (the margin stops two similar machines passing jobs back and forth), or simply in turn. When a head lacks something, it asks the peer with a REQUEST and receives either the object or a `WAITING` promise, after which the peer pushes the object as soon as it has it. The first version polled instead, and under load one head sent 194 315 requests for 1 492 ids until the connection broke. Each link now also has two bounded lanes, one for incoming jobs and one for requests, so heavy asking never stalls incoming work, and a full lane slows its sender through TCP.

## 3.11 What was run and measured

The first check ran two complete nodes on one machine, sharing nothing. The second node learned the first's address only from etcd, finished holding the code, inputs and results it had been sent over sockets, and received the function once for three jobs; after shutdown, etcd was empty. The load-based rule could not be tested this way, since both nodes read the same CPU counters.

Work is produced by a companion project, **pipeIn**. It reads every offset from the engine's header, builds the job functions (`fib`, `factorial`, and `spin` for CPU-heavy jobs), computes every expected result independently in Python, and sends and fetches jobs using the engine's own protocol. Its loads range from `smoke` (3 jobs) through chains, fan-outs and bulk data up to `smokes` (30 000 jobs). Every result so far has matched Python's byte for byte.

| | node 9000 | node 9100 | total |
|---|---|---|---|
| jobs resolved and run | 15 000 | 15 000 | 30 000 |
| at once (input held) | 10 000 | 9 999 | 19 999 |
| after waiting for input | 5 000 | 5 001 | 10 001 |
| wait, mean (ms) | 1 362 | 692 | 1 027 |
| wait, 95th percentile (ms) | 1 956 | 900 | 1 848 |
| requests to the other node | 5 000 | 5 001 | 10 001 |
| `NOT_FOUND` received | 0 | 0 | 0 |
| results pushed to the other node | 153 | 4 981 | 5 134 |

Table 3.1: `smokes`: 30 000 jobs, 10 000 dependent, every dependency crossing between two nodes of one machine. 5.97 s end to end, about 5 000 jobs/s; all results identical to pipeIn's. Each wait cost exactly one request.

### 3.11.1 Bare nodes against the deployment

The planned measurement asks what the platform costs per job. The same engine runs on the same two machines, once as plain processes and once as the Kubernetes deployment, with the same settings. The load, `swarm`, is 5 000 near-free jobs sent to one node and spread between both; `pipein.py bench` starts from empty directories, polls both nodes for results every 200 ms and verifies each one. Preparing it already removed three bottlenecks: the pool dropped jobs when full, the head re-listed its directory for every arrival, and a burst of publishing could outlast the lease.

| run | sent (s) | first (s) | median (s) | last (s) | jobs/s | node 0 / node 1 |
|---|---|---|---|---|---|---|
| bare, 1 | — | — | — | — | — | — |
| bare, 2 | — | — | — | — | — | — |
| Kubernetes, 1 | — | — | — | — | — | — |
| Kubernetes, 2 | — | — | — | — | — | — |

Table 3.2: `swarm`, 5 000 jobs, two machines. *To be completed from the runs.*

## 3.12 Considered and not used

Several established tools could have replaced parts of this design. The table gives the reason each was not taken.

| **alternative** | **why not** |
|---|---|
| Raft, written in-house [21] | needed only for two small keys; a home-made version would be the least-tested part, and etcd already provides it |
| ZooKeeper, Consul | a second heavy service for what a lease already does |
| Gossip (SWIM) [22] | the best option at larger scale; at two nodes, more code for less certainty |
| A DHT (Chord) [23] | unnecessary at this size; rendezvous hashing is the useful part |
| Kafka, RabbitMQ | every job would pass through a third machine — the opposite of moving the job to the data |
| gRPC, Protocol Buffers | they decode messages; the hot path needs raw bytes at fixed offsets (Section 4.2) |
| MPI [7, 8] | fixed ranks in lockstep; here nodes come and go and code arrives at run time |
| Spark, Ray, Dask | more capable, but the thesis would then describe them instead of building the layer |
| Kubernetes as scheduler | decides in seconds, per container; it manages nodes, the engine schedules jobs |
| Shared filesystem | hands the transfer to a system the engine does not control |
| RDMA, DPDK, AF_XDP | help when the network stack is the bottleneck; on 1–2.5 Gb links the link is |

## 3.13 What the engine demonstrates, and what it does not

Running the experiment we get the results seemingly correctly computed, and the results are correct in essence. The metric that we are not covering right here is the one that would tell us how efficiently we are computing one job after arrival. When a job arrives it should wait a very low maximum time until it is computed, while latency was not a priority here, rather throughput was it was not a priority, but if we want to have actual, meainingful results with this system we should be have a way of not only fast forwarding to capable nodes but also a relatively fast way of handling failures and missed packets, which in our set up was handled by TCP sockets and the part of authentification and the issue of corrupted data was further handled by the hashing of

the contents. Row identifiers depend only on row values and must match across heads, compilers and machines. Job identifiers additionally hash the function's compiled bytes, so they are stable only for a fixed toolchain — rebuilding the functions with a different compiler moves every job id while leaving every row id untouched.

The omissions are as follows.

- **Failure handling.** A worker that cannot be reached fails the whole graph. There is no retry, no re-placement, no timeout. As Section 3.4 notes, content-addressed inputs over pure functions already permit recovery by recomputation; what is missing is the bookkeeping to know what to recompute, not the ability to do it.
- **Concurrent dispatch.** Independent jobs placed on different nodes are dispatched one after another. Placement is therefore demonstrated but not yet exploited, and no speedup should be claimed from the current implementation.
- **A single head.** One head owns every job in the graph, which is what makes the single-writer argument of Section 3.6 trivially true. The claim becomes interesting, and requires the membership epoch, only with several heads.
- **Function caching.** The shared object is re-sent with every task, although the job id already contains the hash that would make a per-node cache correct. This is the shortest path from the current prototype to a meaningful reduction in bytes on the wire.
- **Transport selection.** One transport is used for everything. The size-based split between a bulk path and a low-latency path, and the XDP work of Section 4.1, remain design rather than implementation.

What the engine does do complete is the a single authoritative wire format, identity derived from content rather than assigned by a counter, a placement rule that makes destinations known before results exist, and an ownership rule that eliminates the coordination problem instead of solving it. Although however it wouldn't be undesirable to have a built in way to direct execution to manage our own software dynamically and at our command.

## 3.14 Two machines over one 1 Gb link

The run of Table 3.1 placed both nodes on one machine. This section repeats it with one node on each of two machines, joined by a single 1 Gb Ethernet link, with no orchestrator and no etcd: each node is the worker and the head of Section 3.9 started as plain processes, each told the address of the other head with `--peer`, and pipeIn feeding both from the first machine.

### 3.14.1 The setup

| | **machine A** | **machine B** |
|---|---|---|
| address | 192.168.2.1 (`enp34s0`) | 192.168.2.2 |
| processor | AMD Ryzen 7 3700X (8 cores) | AMD Ryzen 9 3900X (12 cores) |
| worker threads | 8 | 14 |
| node | `node-9000`, head on :9000 | `node-9100`, head on :9100 |
| also runs | pipeIn (sends and collects) | — |

Both machines run the same engine source, `engineJava`, each compiled on its own machine; the job functions are compiled once, by pipeIn on A, and travel to B inside the job stream as FUNCTION objects. On A, only the 1 Gb interface carries an address on the 192.168.2.0/24 network, so all traffic between the nodes crosses that one link.

### 3.14.2 Running it

**1. Clear what earlier runs left behind.** A node that already holds a result would serve it at once and spoil the timing, and stale binaries or job files would run something other than the current source. On A:

```
pkill -f JobServerCli; pkill -f 'node --socket'
cd ~/Desktop/RPC/final/engineJava
make clean                                          # node, .o, head classes, head.jar
rm -rf run-9000 run-9100 datasets jobs functions engine.sock
cd ~/Desktop/RPC/final/pipeIn
mv out/bench ~/Desktop/RPC/bench-archive/          # keep the earlier runs
rm -rf out/loads out/functions out/functions.json
```

and on B, `pkill` the same processes and remove any earlier copy of the engine.

**2. Build on A, copy to B, build on B.**

```
cd ~/Desktop/RPC/final/engineJava && ./build.sh
scp -P 22 -r ~/Desktop/RPC/final/engineJava teo@192.168.2.2:~/
ssh -p 22 teo@192.168.2.2 'cd ~/engineJava && make clean && ./build.sh'
```

**3. Start one node on each machine,** each in its own terminal. `run.sh` starts the C worker and the Java head in a fresh `run-<port>/` directory, so every start is an empty node; the node is ready when its head logs `serving`.

```
A:  cd ~/Desktop/RPC/final/engineJava && THREADS=8  ./run.sh 9000 192.168.2.2:9100
B:  cd ~/engineJava                   && THREADS=14 ./run.sh 9100 192.168.2.1:9000
```

**4. Prepare the load and run the bench from A.**

```
cd ~/Desktop/RPC/final/pipeIn
./pipein.py build                       # the job functions -> out/functions/<id>.so
./pipein.py vectors --load smokes       # 20000 input vectors, 3.68 MB
./pipein.py jobs    --load smokes       # 30000 jobs (10000 dependent), 7.7 MB
./pipein.py bench 192.168.2.1:9000 192.168.2.2:9100 \
        --load smokes --alternate --label lan1g --wait 600
```

`bench` first asks both nodes for every result and refuses to start if either already holds one. It then sends the jobs alternately to the two heads — so each copy of `smoke` has its dependent job on the other machine from its producer, and every one of the 10 000 dependencies crosses the link — collects the results from both nodes in rounds, and verifies each against pipeIn's own computation. The run is recorded in `out/bench/lan1g-<time>/` (`bench.log`, `results.csv`, `summary.json`), and `./pipein.py report` lists the runs side by side. Both nodes are restarted (step 3) before the next run.

**The measure.** The figure taken from each run is its *total time*: from the moment the first job is sent to the moment the last result is in, every result verified. Throughput is the number of jobs divided by that time. Results are seen once per collection round, so the times are late by at most one round.

### 3.14.3 The result

All 30 000 results came back and matched pipeIn's, with every dependency resolved across the link: the head holding a dependent job asked the other head for its input once and received it when it existed (Section 3.10). The whole load took 5.234 s, about 5 700 jobs per second, against 5.967 s when both nodes shared one machine. The two runs are not a controlled comparison —

| | one machine (Table 3.1) | two machines, 1 Gb |
|---|---|---|
| jobs (dependent) | 30 000 (10 000) | 30 000 (10 000) |
| bytes sent by pipeIn | 7 742 944 | 7 742 944 |
| sending done (s) | 1.955 | 0.890 |
| first result (s) | 2.566 | 1.484 |
| median result (s) | 3.788 | 3.356 |
| 95th percentile (s) | 5.897 | 4.433 |
| **total time (s)** | **5.967** | **5.234** |
| jobs per second | 5 027.9 | 5 731.3 |
| collection rounds | 5 | 7 |
| results collected from A / B | 15 540 / 14 460 | 15 221 / 14 779 |
| verified | 30 000 of 30 000 | 30 000 of 30 000 |

Table 3.3: `smokes` on two nodes: both on one machine, and one on each machine over a single 1 Gb link.

the second has more cores and more worker threads behind it — and the jobs are nearly free, so the figure measures what the engine costs per job rather than computing power. What it shows is that moving the nodes onto separate machines, with every dependency crossing a real network, did not make that cost worse.

### 3.14.4 The same load over two 10 Gb links

The run was repeated on the same two machines, with the same worker threads (8 on A, 14 on B) and the same `smokes` jobs — the same functions and vectors, so the same job identifiers — but with `engineJava10Gb`, whose heads talk over two channels instead of one. The *control* channel carries REQUESTs, their answers and lone jobs; the *data* channel carries every function and dataset one head sends the other, including each result pushed to the head whose job depends on it.

**The setup.** Each channel has its own direct cable between two 10 Gb ports, and its own subnet, so that each address can be reached through exactly one port and Linux cannot route one channel's traffic through the other's cable. (A first attempt put all four ports on one subnet; the kernel then reached both of the other machine's addresses through the same port, and the data channel carried nothing.)

| channel | A | B | subnet |
|---|---|---|---|
| control | `enp37s0f0`, 10.10.10.11, :9000 | `enp13s0f0`, 10.10.10.12, :9100 | 10.10.10.0/24 |
| data | `enp37s0f1`, 10.10.20.21, :9001 | `enp13s0f1`, 10.10.20.22, :9101 | 10.10.20.0/24 |

```
A:  sudo ip addr flush dev enp37s0f0;  sudo ip addr add 10.10.10.11/24 dev enp37s0f0
    sudo ip addr flush dev enp37s0f1;  sudo ip addr add 10.10.20.21/24 dev enp37s0f1
B:  sudo ip addr flush dev enp13s0f0;  sudo ip addr add 10.10.10.12/24 dev enp13s0f0
    sudo ip addr flush dev enp13s0f1;  sudo ip addr add 10.10.20.22/24 dev enp13s0f1
    (and  sudo ip link set <port> up  for each port)
```

Each head binds its two listeners to its own two addresses and is given both addresses of the other head:

```
A:  THREADS=8  CONTROL_IP=10.10.10.11 DATA_IP=10.10.20.21 PEER_DATA=10.10.20.22:9101 \
        ./run.sh 9000 10.10.10.12:9100
B:  THREADS=14 CONTROL_IP=10.10.10.12 DATA_IP=10.10.20.22 PEER_DATA=10.10.20.21:9001 \
        ./run.sh 9100 10.10.10.11:9000

./pipein.py bench 10.10.10.11:9000 10.10.10.12:9100 \
       --load smokes --alternate --label lan10g --wait 600
```

pipeIn, on A, sends to and collects from the control addresses, so its own traffic shares the control cable and the data cable carries only what the heads send each other.

| | **one machine** | **1 Gb** | **2 × 10 Gb** |
|---|---|---|---|
| sending done (s) | 1.955 | 0.890 | 1.399 |
| first result (s) | 2.566 | 1.484 | 1.941 |
| median result (s) | 3.788 | 3.356 | 3.012 |
| 95th percentile (s) | 5.897 | 4.433 | 4.241 |
| **total time (s)** | **5.967** | **5.234** | **4.595** |
| jobs per second | 5 027.9 | 5 731.3 | 6 529.5 |
| collection rounds | 5 | 7 | 5 |
| verified | 30 000 | 30 000 | 30 000 |

Table 3.4: `smokes`, 30 000 jobs of which 10 000 depend on a result from the other node: two nodes on one machine, on two machines over 1 Gb, and over two 10 Gb links.

| **A's port during the run** | **received** | **sent** |
|---|---|---|
| control, `enp37s0f0` | 8.32 MB | 9.82 MB |
| data, `enp37s0f1` | 1.61 MB | 1.73 MB |

The whole load finished in 4.595 s, against 5.234 s over 1 Gb and 5.967 s with both nodes on one machine — 12 % and 23 % less — and every result was again correct. The counters of the packets show the way we provisioned the machines and jobs: the data port moved about 1.6 MB each way, which is the 10 000 dependency results (about 240 bytes each with their headers) pushed between the heads in both directions, while the control port carried the requests, the answers and all of pipeIn's sending and collecting. Contrary ot what one may expect the communications over the 10Gb nics was not fastest: pipeIn finished sending after 1.4 s, later than over 1 Gb, although the 3.9 MB sent to B would cross a 10 Gb link in a few milliseconds; a head takes in jobs only as fast as its intake lane drains, and a full lane slows the sender through TCP (Section 3.10), so this is probably a case of hte head being the bottleneck here. The gain is in the second half of the run: the median result came 0.3 s earlier and the last one 0.6 s earlier, consistent with the dependency results travelling on their own link, apart from the 15 000 jobs and the polling on the control channel. Each figure is a single run, and differences of a few tenths of a second are within what repeated runs vary by; the ordering of the three totals is what the comparison supports.

# Chapter 4

# Enhancements

The engine of Chapter 3 works, and every packet has to go through the full network stack, which is something that (besides not really ever considering security) is somewhat not needed since the nodes are closely matched and 'isolated'. This chapter collects the technologies that would address each of these:

- **XDP** — deciding what to do with a packet in the network driver, before the kernel spends anything on it (Section 4.1);
- **Protocol Buffers** — a typed, evolvable format for the control messages (Section 4.2);
- **IPC** — the operating-system mechanisms a controller of the engine's own would be built from, and how XDP and io_uring extend them (Section 4.3);
- **KEDA and CRIU** — scaling the cluster on the engine's real load, and moving a process instead of losing it (Sections 4.4 and 4.5);
- and, briefly, the **consensus algorithms** the engine relies on without implementing (Section **??**).

## 4.1 XDP: programmable packet processing

Not only is it not free to move jobs between machines, but rather: every job that crosses pays for the network stack twice, once on each side, and for a job that computes for microseconds that cost can be most of its life, and most important for short lived functions the cost far outweighs local calculations, no matter the burden on the cpu. However when we are talking either about throughput and latency we need to take whatever gains we can get.

**XDP** is an eBPF based high-performance network data path that can 'grab' the packet from the NIC and pass cut throuhg the stack of all the layers to concisely decide what to do with the packet (actions are enumerated below). (eXpress Data Path) attacks that cost where it starts: it lets a small program decide what happens to a packet the moment the network driver receives it, before the kernel spends anything on it. This chapter introduces it through the smallest program worth running.

### 4.1.1 The idea

Without XDP, every received frame is copied into memory by the card, wrapped in an `sk_buff`, passed through IP checks, routing and firewall rules, matched to a socket, and finally copied into the waiting process — all paid *per packet*. XDP adds an early hook to that receive path and lets a user-supplied eBPF program decide the fate of each packet [5, 6]. The hook sits in the network card's driver, just after the interrupt has been handled and *before* any of the memory allocation the network stack needs for itself — the `sk_buff` above — because that allocation is one of the most expensive steps on the path. Figure 4.1 shows where this is: at the far left,

ahead of `alloc_skb` and of every Netfilter table that follows. Placed there, XDP can drop about 26 million packets per second on a single core of commodity hardware [6].

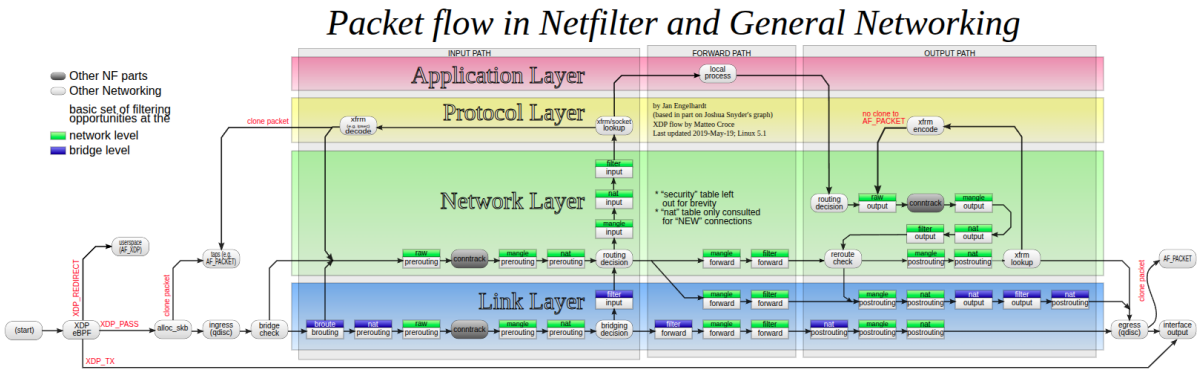



Figure 4.1: The path of a packet through the Linux network stack, from the driver (bottom left) through the link, network and protocol layers to a local process or back out. XDP is the first box after *(start)*: `XDP_PASS` continues to `alloc_skb` and the rest of the stack, `XDP_REDIRECT` leaves for a user-space AF_XDP socket, and `XDP_TX` goes straight to the interface's output. Diagram by Jan Engelhardt, after Joshua Snyder, XDP flow by Matteo Croce; Wikimedia Commons, CC BY-SA.

The program may read and also rewrite the packet's bytes, and when it returns, its action code decides what happens to the packet:

- `XDP_PASS` — hand the frame to the normal stack, as if XDP were not there;
- `XDP_DROP` — discard it silently, before any allocation or wake-up (the basis of line-rate DDoS filtering and of XDP firewalls [13]);
- `XDP_ABORTED` — discard it too, but raise a trace point, for a program that has hit an error;
- `XDP_TX` — send it back out of the same card it arrived on (reflectors, load balancers);
- `XDP_REDIRECT` — send it to another card, another CPU, or straight into a process through the AF_XDP address family (below).

Within limits because of the BPF verifier this can arbitrarily set firewall rules, forwarding even and also editing on the received bytes.

### 4.1.2 eBPF: how the program gets into the kernel safely

XDP programs are **eBPF** programs [17, 16, 14]: restricted C, compiled by `clang` to bytecode for a small virtual machine inside the kernel. To be able to use an ebpf program a **verifier** checks the program before it may run in kernel space at all: it proves that the program terminates and that every memory access is inside the packet or a map, and refuses it otherwise, which otherwise would mean access to random kernel memory. In its first versions this meant no loops and no global variables whatsoever; kernels since 5.3 accept loops whose bound the verifier can establish, and global variables are kept in maps behind the scenes, but the principle is unchanged — nothing is loaded that could read out of bounds or run for ever. The **JIT** then compiles it to native code. **Maps** — key/value tables shared with user space — hold everything the program remembers between packets, so user space configures and observes a running program by reading and writing maps, without reloading it.

### 4.1.3 Hello, XDP

The program below counts every IPv4 packet by protocol and drops pings; everything else passes untouched:

```
// hello_xdp.bpf.c
#include <linux/bpf.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/in.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

struct {                                    // a map: one counter per IP protocol number
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 256);
    __type(key, __u32);
    __type(value, __u64);
} counts SEC(".maps");

SEC("xdp")
int hello(struct xdp_md *ctx)
{
    void *data = (void *)(long)ctx->data;
    void *end  = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > end) return XDP_PASS;          // bounds check, or no load
    if (eth->h_proto != bpf_htons(ETH_P_IP)) return XDP_PASS;

    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > end) return XDP_PASS;

    __u32 proto = ip->protocol;                              // 1 ICMP, 6 TCP, 17 UDP
    __u64 *n = bpf_map_lookup_elem(&counts, &proto);
    if (n) __sync_fetch_and_add(n, 1);

    return proto == IPPROTO_ICMP ? XDP_DROP : XDP_PASS;
}

char _license[] SEC("license") = "GPL";
```

`SEC("xdp")` marks the entry point and `SEC(".maps")` declares the map. Each header is reached from the one before and compared with `end` *before* it is read: remove any of the three checks and the verifier refuses to load the program. The counter is added atomically because the program runs on every receiving CPU at once. The `GPL` licence is required by the kernel helpers it calls.

Compiling, attaching to the loopback interface and trying it needs only `clang`, `libbpf`'s headers and root; `ip` loads the object itself:

```
clang -O2 -g -target bpf -c hello_xdp.bpf.c -o hello_xdp.o
sudo ip link set dev lo xdpgeneric obj hello_xdp.o sec xdp    # attach
ping -c 3 127.0.0.1                                          # 100% loss: dropped in XDP
sudo bpftool map dump name counts                            # key 1 (ICMP): 3
sudo ip link set dev lo xdpgeneric off                       # detach: ping works again
```

Turning this into something useful changes little: a hash map keyed by whatever the program should route on, and a `return bpf_redirect_map(...)` instead of `XDP_PASS`. User space then moves traffic by writing one map entry.

### 4.1.4 Attachment modes and AF_XDP

XDP needs support in the card's driver, and not every driver has it. Where it is missing there is a fallback: the `xdpgeneric` above is *generic* mode, in which the program runs inside the network stack after the kernel has built its `sk_buff`. That works on any interface — loopback, a

container's `veth` — with the same program and the same verdicts, but saves little. *Native* mode (`xdpdrv`) runs it inside the driver, before any allocation, and is where the speed is; the Intel X540 cards of Section 4.1.6 (`ixgbe`) support it. *Offloaded* mode goes one step further and runs the program on the card itself, taking it off the CPU entirely.

To bring packets into a process rather than the stack, the program redirects them into an **AF_XDP** socket. Its *UMEM* is a block of the process's memory, cut into frames, that in native mode the card writes into directly — no copy. For a start this is the way that this form of 'IPC' could be resumed. And for performance four rings shared with the kernel move frames around: the process hands empty frames to the kernel on FILL, receives full ones on RX, sends on TX and gets sent frames back on COMPLETION. A frame being processed cannot be refilled, so the number of frames bounds the work in flight: when all are busy the card drops. The price is everything the stack did: AF_XDP delivers raw frames, not exactly a problem when we have taken care of the design of the atual packages being circulated in our system.

### 4.1.5 More usages of XDP for the system

```
clang -O2 -g -target bpf -I/usr/include/$(uname -m)-linux-gnu -c prog.c -o prog.o

ip netns add test
ip link add veth0 type veth peer name veth1 netns test
ip addr add 10.0.0.1/24 dev veth0;           ip link set veth0 up
ip -n test addr add 10.0.0.2/24 dev veth1;   ip -n test link set veth1 up
ip link set dev veth0 xdp obj prog.o sec xdp     # attach    (... xdp off: detach)
ip netns del test                                 # clean up
```

**Drop everything.** The kernel calls the function once per packet, and here it returns a verdict without looking at the packet at all; `ip netns exec test ping -c 3 10.0.0.1` then loses every packet.

```
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>

SEC("xdp")
int xdp_drop_all(struct xdp_md *ctx)
{
    return XDP_DROP;
}

char LICENSE[] SEC("license") = "GPL";
```

`SEC("xdp")` puts the function in an ELF section named `xdp`, which is how the loader finds it; the `license` section unlocks the helpers that are available to GPL programs only.

**A protocol of our own.** The most important application of the technology for our programs however is that we can have access to the raw contents of the messages. Nothing requires IP after the Ethernet header. Using EtherType `0x88B5`, reserved for local experiments, a sender can put any bytes there — here a 16-byte message of its own design, `[magic | type | 11 bytes]` — and an XDP program can read them before the kernel would even know what they are:

```
#define ETH_P_CUSTOM 0x88B5
#define MY_MAGIC     0xC0FFEE01

struct my_msg { __be32 magic; __u8 type; __u8 data[11]; } __attribute__((packed));

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 256);
```

```
    __type(key, __u32);
    __type(value, __u64);
} type_count SEC(".maps");

SEC("xdp")
int xdp_custom(struct xdp_md *ctx)
{
    void *data     = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end)            return XDP_PASS;
    if (eth->h_proto != bpf_htons(ETH_P_CUSTOM)) return XDP_PASS;   /* not ours  */

    struct my_msg *msg = (void *)(eth + 1);
    if ((void *)(msg + 1) > data_end)            return XDP_DROP;   /* too short */
    if (msg->magic != bpf_htonl(MY_MAGIC))       return XDP_DROP;   /* garbage   */

    __u32 key = msg->type;
    __u64 *cnt = bpf_map_lookup_elem(&type_count, &key);
    if (cnt)
        __sync_fetch_and_add(cnt, 1);

    bpf_printk("msg type %d, data %x %x", msg->type, msg->data[0], msg->data[1]);
    return XDP_DROP;                                                 /* consumed  */
}
```

This for a start could mean that we can validate and or pass through only our 'magic number labeled' messages and if we want the lowest latency possible it's easy to set up the receiving end on a machine of our cluster. Further, we can route on this level either based on external commms about loads and metadata for the nodes, or we can route towards different receiving programs based on hte content of the package. This should be no different than the low level handling of the bytes by the worker process.
`packed` removes the padding, so the struct is exactly the $4 + 1 + 11$ bytes the sender writes. The sender is a raw packet socket in the namespace:

```
#!/usr/bin/env python3
import os, socket, struct, sys

msg_type = int(sys.argv[1]) if len(sys.argv) > 1 else 1
s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW)
s.bind(("veth1", 0))
eth = b"\xff" * 6 + s.getsockname()[4] + struct.pack("!H", 0x88B5)  # broadcast, our MAC
s.send(eth + struct.pack("!IB", 0xC0FFEE01, msg_type) + os.urandom(11))

ip link set dev veth0 xdp obj xdpCustom.o sec xdp
cat /sys/kernel/tracing/trace_pipe                # second terminal: bpf_printk output
ip netns exec test python3 send.py 7
bpftool map lookup name type_count key 7 0 0 0    # the count for type 7
```

**Calling more code.** An interface runs one XDP program, so more logic means either functions inside it or a jump to another program. Ordinary C functions work: `static __always_inline` pastes the body into the caller, `static __noinline` makes a real BPF-to-BPF call, at most eight deep. A *tail call* instead jumps into a second program stored in a `PROG_ARRAY` map; like `exec` it never returns, the second program's verdict is final, and if the slot is empty execution simply continues after the call. A chain may hold at most 33 of them.

```
struct {
    __uint(type, BPF_MAP_TYPE_PROG_ARRAY);
    __uint(max_entries, 4);
```

```
    __type(key, __u32);
    __type(value, __u32);
} jump_table SEC(".maps");

SEC("xdp")
int xdp_stage2(struct xdp_md *ctx)                  /* runs only for our frames */
{
    bpf_printk("stage2: custom frame");
    return XDP_DROP;
}

SEC("xdp")
int xdp_entry(struct xdp_md *ctx)                   /* attached to the interface */
{
    void *data     = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;
    struct ethhdr *eth = data;

    if ((void *)(eth + 1) > data_end)
        return XDP_PASS;
    if (eth->h_proto == bpf_htons(0x88B5))
        bpf_tail_call(ctx, &jump_table, 0);         /* jump to slot 0 */
    return XDP_PASS;                                /* not ours, or slot 0 empty */
}
```

`ip link` cannot fill the jump table, so this one is loaded with `bpftool`, which pins both programs and the map in the BPF filesystem:

```
P=/sys/fs/bpf/xdptail
bpftool prog loadall xdpTail.o $P type xdp pinmaps $P/maps
bpftool map update pinned $P/maps/jump_table key 0 0 0 0 value pinned $P/xdp_stage2
bpftool net attach xdp pinned $P/xdp_entry dev veth0
bpftool net detach xdp dev veth0;  rm -r $P                  # clean up
```

(Older `bpftool` versions pin programs under their section names; the listing of `$P` shows which.) The alternative without tail calls is libxdp's dispatcher, which runs separately compiled programs in turn for as long as each returns `XDP_PASS`: `xdp-loader load -m native veth0 first.o second.o`. And this is the way that we could use the tech to enhance our products performance.

### 4.1.6 Evaluation of performance gains

XDP was not used in the running examples, however we are able to see for ourselves the performance gained via a very simple experiment. In a set up similar to the last engine's experiment run, one 64-byte UDP message is bounced back and forth $N$ times between the two machines of Section 3.14.4, over their two 10 Gb links side by side (Intel X540, driver `ixgbe`, which supports native XDP). On one link the far machine answers from an ordinary UDP echo server, through the whole network stack; on the other an XDP program answers from inside the driver with `XDP_TX`, and the stack never sees the packet. The same client measures both, and for both pcs we had an Intel x540 T2 NIC that had dual 10Gb capabilities.

| **link** | **local (client)** | **remote (server)** | **remote answers with** |
|---|---|---|---|
| normal | `enp37s0f0` 10.10.10.11 | `enp13s0f0` 10.10.10.12 | `echo_server.py` |
| xdp | `enp37s0f1` 10.10.20.21 | `enp13s0f1` 10.10.20.22 | `xdp_echo.o` |

The addresses are those of the engine's two channels, set up the same way; both `ping 10.10.10.12` and `ping 10.10.20.22` must answer before starting.

**The XDP echo.** For UDP packets to port 9000 the program swaps the MAC addresses, the IP addresses and the ports, and returns `XDP_TX`, which sends the packet straight back out of the port it came in on. Anything else — ARP above all — is passed to the kernel. This ofc would make the gain more dramatic.

```
SEC("xdp")
int xdp_echo(struct xdp_md *ctx)
{
    void *data     = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end || eth->h_proto != bpf_htons(ETH_P_IP))
        return XDP_PASS;                                    /* ARP etc. */
    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end || ip->ihl != 5 || ip->protocol != IPPROTO_UDP)
        return XDP_PASS;                                    /* UDP, no options */
    struct udphdr *udp = (void *)(ip + 1);
    if ((void *)(udp + 1) > data_end || udp->dest != bpf_htons(9000))
        return XDP_PASS;                                    /* not our port */

    unsigned char mac[ETH_ALEN];
    __builtin_memcpy(mac, eth->h_dest, ETH_ALEN);
    __builtin_memcpy(eth->h_dest, eth->h_source, ETH_ALEN);
    __builtin_memcpy(eth->h_source, mac, ETH_ALEN);

    __be32 addr = ip->saddr;   ip->saddr = ip->daddr;     ip->daddr = addr;
    __be16 port = udp->source; udp->source = udp->dest;   udp->dest = port;

    return XDP_TX;
}
```

**The normal echo and the client.** The ordinary server is a simple Python script bound to 10.10.10.12:9000. The client, on the local machine, warms each link up with 500 round trips (ARP, caches), then times $N$ more, one at a time, and prints for each link the total time, the mean, median and 99th-percentile round trip, and the ratio of the two totals.

```
# echo_server.py (remote)
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(("10.10.10.12", 9000))
while True:
    data, addr = s.recvfrom(2048)
    s.sendto(data, addr)
```

```
# pingpong.py (local), per link
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.connect(addr);  s.settimeout(1.0)
for _ in range(500): s.send(MSG); s.recv(2048)              # warm-up
start = time.perf_counter()
for _ in range(n):
    t0 = time.perf_counter_ns()
    s.send(MSG); s.recv(2048)
    rtts.append((time.perf_counter_ns() - t0) / 1000)       # microseconds
total = time.perf_counter() - start
```

### Running it.

```
remote:  clang -O2 -g -target bpf -c xdp_echo.c -o xdp_echo.o        # + the -I of above
         sudo ip link set dev enp13s0f1 xdpdrv obj xdp_echo.o sec xdp   # native mode
         ip link show dev enp13s0f1                                     # shows prog/xdp
         python3 echo_server.py
```

```
local:   python3 pingpong.py 10000
remote:  sudo ip link set dev enp13s0f1 xdpdrv off                    # afterwards
```

Two things commonly get in the way. A firewall on the remote machine blocks the normal link (`ufw allow 9000/udp`) but not the XDP one, since XDP answers before the firewall is reached. And `ixgbe` accepts a native XDP program only at a standard MTU of 1500 and with no other program attached.

**What it measures.** The client code is identical for both links, so any difference comes from the remote side: a packet that climbs the stack to a process and back down again, against one turned around in the driver. Two cautions apply to the comparison. The normal server is Python, which widens the gap; a C echo server on 10.10.10.12:9000 would give the stricter comparison. This is not to be underestimated, and could account for the difference alone however further the nics used seem to have higher latencies than the one hosted on the motherboard (1Gb), and is something that could be affected by pcie slot condition or the routing from that to the cpu, or the indirect routing trhough the motherboard's chipset.

| run | normal | xdp | ratio |
|---|---:|---:|---:|
| 10 000 round trips | 303.24 µs (3.03 s) | 94.30 µs (0.94 s) | 3.2× |
| 1 000 000 round trips | 310.75 µs (310.7 s) | 160.45 µs (160.5 s) | 1.9× |
| 10 000, xdp link only | — | 168 µs (1.68 s) | — |

Table 4.1: Mean time per round trip of a 64-byte UDP message, with the total time of each run in brackets; the ratio is the normal link's total over the XDP link's.

The XDP link was faster in every run: 3.2 times over 10 000 round trips and 1.9 times over a million. The normal link was steady, at 303 to 311 µs per round trip; the XDP link varied from run to run, between 94 and 168 µs. On the XDP link the packet is turned around inside the remote machine's driver, before the network stack and before any process, so the remote side's share of the round trip — the stack in both directions and the Python server — is gone; the client's share, the same on both links, remains. Both figures are nevertheless far above what one round trip over a direct 10 Gb cable needs, so something other than the work being measured is delaying every packet. The likeliest causes are interrupt moderation on the cards, which holds received packets back to deliver them in groups, and the processors' power-saving sleep states, from which a core takes time to wake for each packet; both would also explain the spread of the XDP figures. Neither has been tested yet. Although testing for this is incomplete I think that the capabilities are evident and the general ease of use explain the number of usages of it on enterprise software for either routing intense jobs and or loggin over distributed systems usually in the wild deployed over kubernetes and in the cloud.

## 4.2 Protocol Buffers

**Protocol Buffers** (protobuf) [20] are Google's language for describing structured messages, and the compact binary format packages can travel in when optimised for thatt. The compiler `protoc` generates the .proto format for each message, and for each language, the code that writes and reads it which serves as the link on multiple stack microservice setups usually. On the wire each field is identified by a *number* — never by its name or position — and this is what makes the format compact and lets a schema change without breaking older programs.

### 4.2.1 Hello, protobuf

```
// reading.proto
syntax = "proto3";
```

```
message Reading {
  string          sensor      = 1;
  sint32          temperature = 2;   // may be negative
  uint32          pressure    = 3;
  repeated uint32 samples     = 4;
}
```

`protoc -python_out=. reading.proto` generates `reading_pb2.py`; using it is two calls:

```
import reading_pb2 as pb
r = pb.Reading(sensor="kitchen", temperature=-3, pressure=1013, samples=[3, 300])
data = r.SerializeToString()                 # 19 bytes
assert pb.Reading.FromString(data) == r
```

### 4.2.2 The 19 bytes

Each field is written as a **key** and a **value**. The key is `(field_number << 3) | wire_type`, and the wire type only says how to find where the value ends: 0 is a *varint*, 2 a length followed by that many bytes, 1 and 5 fixed 8 and 4 bytes. A varint spends seven bits per byte, low bits first, with the top bit set while more bytes follow.

```
  0A 07 6B 69 74 63 68 65 6E   key 0A = (1<<3)|2: field 1, length 7    "kitchen"
  10 05                        key 10 = (2<<3)|0: field 2, varint      -3  (ZigZag: 5)
  18 F5 07                     key 18 = (3<<3)|0: field 3, varint      1013
  22 03 03 AC 02               key 22 = (4<<3)|2: field 4, length 3    [3, 300]
```

Three details are worth reading off the bytes.

- **Varints.** 1013 is `F5 07`: the low seven bits `1110101` with the continuation bit set, then `7`; 300 is likewise `AC 02`. Small numbers cost one byte.
- **Negative numbers.** A plain `int32` would write $-3$ as a 64-bit two's complement varint, ten bytes. `sint32` first applies *ZigZag* ($0, -1, 1, -2, 2, \ldots \mapsto 0, 1, 2, 3, 4, \ldots$), so $-3$ becomes 5 and costs one byte.
- **Nothing else.** There are no names, no field order to respect, no total length and no type: `protoc -decode_raw` can show the numbers and values, but only the `.proto` says what they mean. A field equal to its default (0, empty) is not sent at all, so "absent" and "zero" are the same thing.

### 4.2.3 Schema evolution

Suppose a later version adds `uint32 battery = 5`. An older reader meets key `28` $= (5 \ll 3)|0$, does not know field 5, sees wire type 0 and skips one varint. A newer reader given an older message finds no field 5 and reads 0. Both directions work as long as a field number is never reused; a removed field's number is kept out of use with `reserved 5;`. This is what lets the programs of a running system be upgraded one at a time rather than all at once.

### 4.2.4 How schema-based encodings work: Protocol Buffers, Thrift and Avro

Protocol Buffers belongs to a small family of binary formats that describe data with a schema, together with Apache Thrift and Apache Avro. How each of them writes a record into bytes decides which changes to its schema it can survive, and Kleppmann explains all three through one small record [19, 18]: a person with a name, a favourite number and a list of interests, `{"userName": "Martin", "favouriteNumber": 1337, "interests": ["daydreaming", "hacking"]}`.

**Protocol Buffers.** The schema gives every field a tag number:

```
message Person {
  required string user_name        = 1;
  optional int64  favourite_number = 2;
  repeated string interests        = 3;
}
```

The encoded record is just the 33 bytes in all. Each field opens with one byte that holds its tag number and its wire type (Section 4.2.3). A string is then followed by its length and its UTF-8 bytes, an integer by its value as a varint, this can enhance performance on cases where like ours we had very small jobs being transmitted where both the data and the .sos being passed around were sufficiently small (even small enough to fit in a single package of UDP, which could be paired with XDP).

Optionally, repeated fields are encoded alike, differing only in how often their tag may appear, so a field can be turned from optional into repeated or back; a parser expecting a single value and finding several keeps the last. `required` adds a check on top, and changing it risks a runtime error whenever sender and receiver disagree — one reason proto3, used above, no longer has it. A field with no value takes no bytes at all, so an optional or repeated field can be removed from the schema, but its tag must never be given to another field, because data written earlier may still carry it. A new field can be added under a new tag: an old parser does not know its name, but the type bits tell it how many bytes to skip to reach the next field. And since names never reach the wire, a field can be renamed freely; only its tag number is permanent.

### 4.2.5 Framing, and where protobuf fits

Because a message carries no length and no type, a stream of them needs framing: usually a varint length before each message (`13` before the 19 bytes above), and a wrapper message with a `oneof` when several types share a connection. Decoding also means walking the keys to reach a field. Protobuf therefore suits messages that are small, typed and long-lived across versions — service interfaces, configuration, stored records — and not paths that must read data in place with no decoding, where fixed layouts such as FlatBuffers or Cap'n Proto are used instead.

### 4.2.6 Where it would fit in the engine

We could have used a ready framework or just applied a custom mapping tool on each receiving end of the system, but in the engine, protobuf belongs on the *control* path and not on the data path. Job submission, status queries, membership and the head–worker channel are small typed messages that change as the engine grows, which is exactly what protobuf is for: one `.proto` would generate the Java head's, the C worker's and pipeIn's Python code, and old and new heads could keep talking during an upgrade, if this approach was taken. Job payloads should stay in the engine's own flat objects (Section 3.3.1), which the worker reads in place with no decoding at all. This approach was taken partly because in a not loosely coupled system like ours we would expect that the usages would revolve around computation of jobs that have a certain affinity, and in so doing only the data would end up being passed around on the network. Of course larger data could be routed through TCP for authentication and 'security' in the sense of not halting execution or harming results while waiting missing pieces of data, it 'd be interesting to see if there'd be any such losses and if it would be instead possible to run this all on UDP, perhaps with TCP like packet ensembling over UDP on the XDP side.

## 4.3 IPC: the controller's mechanisms

A controller of our own is a process that could start other processes, gives them channels, hands them work and notices when they die. Each of those is a Linux system call; and they could

be used to provide lower application size and debloating, following Kerrisk's book on the linux interface for system calls we have multiple choices on how to manage the lifecylcle of hte processes we need and how to dynamically set up the IPC channels needed for the transfer of code and data. [1].

### 4.3.1 Processes, signals and the lifecycle

A process is created with `fork`, given a new program with `execve`, and after it exits stays a dormant until its parent collects it with `waitpid`. Open descriptors and the signal mask survive both `fork` and `exec`, which leads to three rules for any controller:

- mark descriptors a child must not inherit *close-on-exec*, or workers keep each other's channels open and never see end-of-file;
- restore the signal mask in the child before `exec`;
- use `PR_SET_PDEATHSIG` so a child dies with its controller.

Child exits are best handled in the event loop: block `SIGCHLD`, read it from a `signalfd`, and call `waitpid` in a loop, since several exits can arrive as one signal. Spawning and reaping a worker:

```
socketpair(AF_UNIX, SOCK_STREAM, 0, sv);  fcntl(sv[0], F_SETFD, FD_CLOEXEC);
pid_t parent = getpid(), pid = fork();
if (pid == 0) {                               /* child */
    prctl(PR_SET_PDEATHSIG, SIGTERM);
    if (getppid() != parent) _exit(1);
    sigprocmask(SIG_SETMASK, &oldMask, NULL);
    /* ENGINE_FD=sv[1] in the environment */  execl("./worker", "worker", (char *)NULL);
    _exit(127);
}
close(sv[1]);
/* later, when the signalfd is readable: */
while ((p = waitpid(-1, &status, WNOHANG)) > 0) { requeueJobsOf(p); restartSlotOf(p); }
```

### 4.3.2 Threads as workers

Processes give isolation; threads give sharing, which makes handing a job to a thread as cheap as passing a pointer — but a crashing thread takes the whole process with it. Threads are created with the stop signals blocked so that only the main thread receives them. So ofcourse a process could be heavier than a userspace posix thread on C, but however we can apply isolation mechanisms on it based on the container isolation principles and of course we have the advantage that a process crashing minimally affects its siblings.

```
static void *workerThread(void *arg) {
    for (;;) {
        pthread_mutex_lock(&pool.mu);
        while (pool.count == 0 && !pool.stopping) pthread_cond_wait(&pool.notEmpty, &pool.mu);
        if (pool.count == 0) { pthread_mutex_unlock(&pool.mu); return NULL; }
        struct task t = pool.q[pool.head];  pool.head = (pool.head + 1) % QCAP;  pool.count--;
        pthread_cond_signal(&pool.notFull);
        pthread_mutex_unlock(&pool.mu);
        t.fn(t.in, t.inLen, t.out, t.cap);            /* the job, outside the lock */
    }
}
```

### 4.3.3 The IPC mechanisms

Linux offers two families. System V objects are found by numeric keys, live until removed explicitly, and cannot be waited on with `epoll`. POSIX objects have names and, where possible, file descriptors, so queues can join an event loop and semaphores avoid the kernel when uncontended. The real choice, though, is between channels the kernel buffers and shared memory:

| mechanism | copies | syscalls, busy | pollable | the other side dies |
|---|---|---|---|---|
| pipe, FIFO, UNIX socket | 2 | 2 | yes | EOF; writer gets `EPIPE` |
| TCP | 2 | 2 | yes | EOF or reset; crosses machines |
| POSIX message queue | 2 | 2 | yes (Linux) | nothing: the queue stays |
| System V message queue | 2 | 2 | no | nothing, until `IPC_RMID` |
| shared-memory ring | 1 or 0 | 0 | no | nothing, unless supervised |
| + futex / semaphore | — | to sleep / wake | no | robust mutex: `EOWNERDEAD` |
| eventfd, signal | — | 1 | yes / via `signalfd` | — |

Right now the engine uses a socket from head to worker, renamed files back, and TCP between machines.

### 4.3.4 Configuring the channels

File descriptors are used for the setting up of the channels, the abstraction that linux uses for processes to have a common endpoint to communicate: i.e. read and write to and from eachother. A `socketpair` gives each worker a private stream with no name to clean up. A POSIX shared-memory segment with process-shared semaphores inside it gives a fast ring for bulk traffic (store indices in it, never pointers). And a POSIX message queue carries small commands, with its descriptor waited on in the same `epoll` loop as everything else.

```
int fd = shm_open("/engine-ring", O_CREAT | O_EXCL | O_RDWR, 0600);
ftruncate(fd, sizeof(struct Ring));
struct Ring *r = mmap(NULL, sizeof *r, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
sem_init(&r->items, 1, 0);  sem_init(&r->slots, 1, CELLS);
/* producer: sem_wait(slots); write cell[tail % CELLS]; tail++; sem_post(items) */
/* consumer: sem_wait(items); use  cell[head % CELLS]; head++; sem_post(slots) */
```

### 4.3.5 Controllers and client–server pairs

A controller either passes messages to its workers and waits on all their channels with `epoll`, or shares a state table with them in memory, guarded by semaphores. Pairing a server with clients it does not know in advance always uses a well-known meeting point (a FIFO path, a queue key, a listening socket) followed by a private channel per client; for sockets, `accept` does this automatically. The engine's worker is the simplest case: a server for exactly one client. This is a somewhat decoupled way and one that could be used to launch more threads/processes running the code.

### 4.3.6 The worker's `main`

The worker's start-up applies most of the above in order: it installs its signal handlers (without `SA_RESTART`, so a stop request interrupts `accept`), reserves its two memory arenas, starts the pool with signals blocked, accepts its one head and removes the socket, then reads objects — checking each declared length before reading further — until the head leaves, when it stops the pool and releases its memory.

### 4.3.7 The biggest gain to be had

However, the biggest gain to be had would be a shared memory set up of the processes IPC channel, that could be further enhanced via buffer rings when using shm and mmap to pass the memory array into each of the processes and also it could be a message queue that would make the code much safer since an element taken from the queue would be ready to run since it would carry all the info. Additionally since a queue is exactly what we have the lack of the necessity of

having to run code reading and making sure we have read exactly such bytes or understanding the end of the byte array would make the code much better and perhaps performant.

What we would end up with would be somewhat of a small copy of kubernetes perhaps. but this would make the system have the latency and depending on the set up it could be easily made to be scalable, albeit without the dynamic and springing development environment kubernetes is offering.

| **Kubernetes** | **a controller of our own** |
|---|---|
| API server + etcd | the member table in shared memory |
| `kubectl` | `enginectl`: mailbox + `SIGUSR1` |
| kubelet starts a container | `fork` + `exec` |
| restart on exit | `SIGCHLD`, `waitpid`, restart — and requeue its jobs |
| liveness probe | the process table; `EOWNERDEAD` |
| Service, DNS | inherited socket; named shm endpoint |
| HPA / KEDA on CPU | scaling on queue depth, read directly |
| pod network | shared rings and heap |

The point is not to replace Kubernetes but to use each at its own level: Kubernetes places one pod per machine, and inside it the controller runs the node job by job. The costs are real — it works only within one machine, isolation is weaker, it relies on Linux-specific features, and supervision bugs become ours to fix. For `engineJava` the step is small: create the head–worker channel with `socketpair`, start both processes, and restart both if either dies.

### 4.3.8 XDP and io_uring, together

AF_XDP (Section 4.1.4), shared-memory rings and io_uring is a syscall in linux made to take the overhead of syscalls and minimize it. As AF_XDP does this between the network card and a process, a shared-memory ring between two processes, and io_uring between a process and the kernel for everything else — files, sockets, timers — by batching many requests into one system call, or none with `SQPOLL`. Combined, they give a node in which job frames never meet the ordinary network stack:

```
NIC --> XDP: bad header -> DROP;  heads' TCP, etcd -> PASS;  job -> REDIRECT to AF_XDP
        v
ingress   payload from the UMEM frame, copied once into the shared store
        v  shared-memory ring (no syscall while busy)
executors run the job in place
        v  shared-memory ring
collector io_uring: write, rename, send results -- one system call per batch
```

How this works is by creating a queue of syscalls waiting to be finalized, after which event they'll be read by the caller.

io_uring replaces one system call per operation with two rings in memory shared with the kernel (Figure 4.2): the application queues requests at the tail of the submission queue, the kernel takes them from its head and posts each result to the completion queue, so many operations cost a single system call, or none [15]. In the worker every finished job now costs four calls — `open`, `write`, `close`, `rename` into `datasets/` — and every object is read off the head's socket with blocking `read`s. With io_uring the four would become one linked chain, the chains of many jobs submitted together, and a multishot receive would replace the reads with one request that keeps delivering; a burst of near-free jobs such as `smokes` would pay for its I/O in batches, and the pool threads would go back to running jobs instead of waiting on the kernel. In such a syscall heavy system like ours we would benefit greatly from this since all of the operations would be sequential. A case where the $\text{io}_u ring needs special care since it cannot guarantee the order of completion, only that of submition to the kernel.$

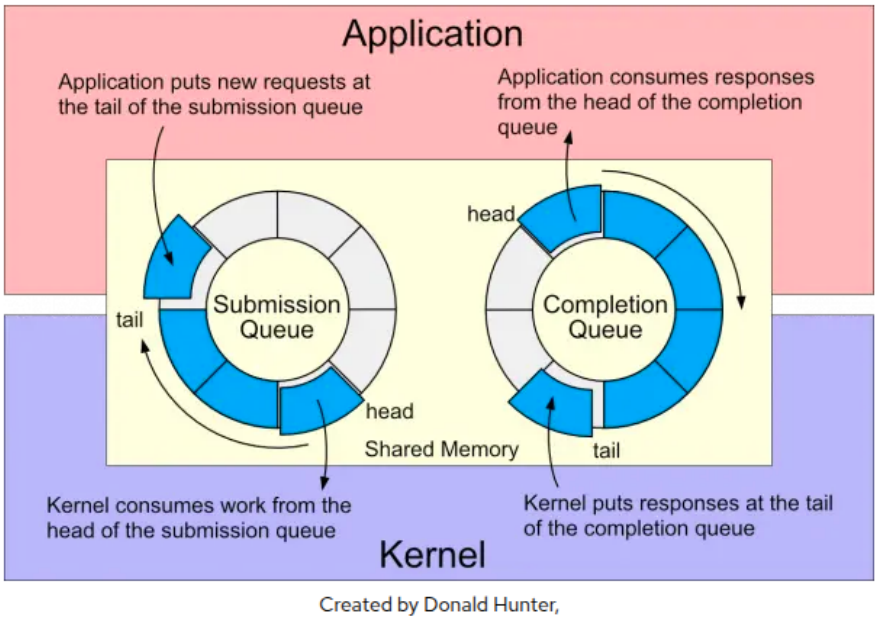



Figure 4.2: io_uring's submission and completion queues, shared between the application and the kernel. Diagram by Donald Hunter [15].

## 4.4 KEDA: scaling on the engine's own load

KEDA already scales the deployment of Chapter 2, but on CPU, which is only a proxy for the engine's load: with one busy thread per pod a pod reads 100 % at most, and a 70 % target could only grow the cluster from two pods to three.

**Scale heads and workers apart.** A worker is limited by CPU and that's governed by the speed of the head feeding it, while mostly a head by the number of jobs and connections it handles; scaling them together as whole nodes over-provisions one to satisfy the other. The project `engineHeadWorkers` splits them — a pod is either a head or a worker — with the worker listening on TCP, returning each result on its job's connection, and registering itself in etcd under a lease. The heads are a StatefulSet with a volume each, the workers a Deployment, and each gets its own ScaledObject. Run as processes, a killed worker's jobs were re-sent and every result arrived, and a new worker received work within half a second.

**Scale on the queue, not the CPU.** Besides CPU, KEDA can read a number from an HTTP endpoint, a Prometheus query, or an external scaler it calls over gRPC. The heads — or a controller of the node (Section **??**) — already know how many jobs are queued and how many threads are running them. Publishing that one number would let the cluster add pods when the queues say the machines are full, while a controller inside each pod adjusts its worker processes within seconds.

```
triggers:                    # proposed: each head serves its load on :9180
  - type: metrics-api
    metadata:
      url: "http://engine-0.engine.engine.svc:9180/load"
      valueLocation: "queuedPerThread"
      targetValue: "2"
```

## 4.5 CRIU: moving a process instead of losing it

When Kubernetes restarts a container, or the engine re-sends a dead worker's jobs, a job that was halfway through starts again from nothing. **CRIU** (Checkpoint/Restore In Userspace) freezes a process tree and writes its complete state to disk — memory, registers, open files, and with its TCP repair mode even established connections — then restores it later, on the same machine or another, continuing where it stopped. Kubernetes exposes checkpointing through the kubelet (container checkpointing, built on CRIU); restoring is not yet a first-class operation and is done by building an image from the archive.

As a result we could have fault tolerance and with relatively low latency, additionally we are able to dynamically add and take out nodes. For example in user lended hardware that run on each users home, we could easily send out a package that initiates the deployment there, and then launches one instanceof the head/worker node, if for some reason the user decided to turn it off CRIU could take it elsewhere literally and figuratively.

```
sudo criu dump    -t <pid> -D ckpt/ --shell-job --tcp-established   # freeze and save
sudo criu restore -D ckpt/ --shell-job --tcp-established            # continue, here or elsewhere
```

# Bibliography


[1] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press, 2010.

[2] B. Burns, J. Beda and K. Hightower, *Kubernetes: Up and Running*, O'Reilly Media.

[3] B. Burns, *Designing Distributed Systems*, O'Reilly Media, 2018.

[4] N. Poulton, *The Kubernetes Book*, self-published.

[5] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern and D. Miller, "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel", *ACM CoNEXT*, 2018.

[6] Wikipedia, "Express Data Path". `https://en.wikipedia.org/wiki/Express_Data_Path`

[7] S. Stork (HLRS, Stuttgart), "Open MPI", Interactive European Grid, MPI Grid User Course, Dublin, 2007. `https://indico.cern.ch/event/16806/contributions/258589/attachments/205996/289029/openmpi.pdf`

[8] R. H. Castain and J. M. Squyres, "The ABCs of Open MPI: Decoding the Alphabet Soup of the Modern HPC Ecosystem (Part 1)", EasyBuild Tech Talks, 23 June 2020. `https://easybuild.io/files/easybuild-tech-talks/easybuild_tech_talks_01_OpenMPI_part1_20200623.pdf`

[9] Apache APISIX, "Stream Proxy", *Apache APISIX Documentation*. `https://apisix.apache.org/docs/apisix/stream-proxy/`

[10] H. Liu, "Get started with XDP", *Red Hat Developer*, 1 April 2021. `https://developers.redhat.com/blog/2021/04/01/get-started-with-xdp`

[11] eunomia-bpf, "eBPF Tutorial by Example 21: Programmable Packet Processing with XDP", 2023. `https://eunomia.dev/tutorials/21-xdp/`

[12] N. Dhandala, "How to Set Up XDP for High-Performance Packet Processing on Ubuntu", *OneUptime Blog*, 2 March 2026. `https://oneuptime.com/blog/post/2026-03-02-set-up-xdp-high-performance-packet-processing-ubuntu/view`

[13] Oracle Linux Blog, "eBPF firewall with XDP". `https://blogs.oracle.com/linux/ebpf-firewall-xdp`

[14] Kernel Recipes, *Document library* (talks on the Linux kernel, its networking and BPF). `https://archives.kernel-recipes.org/document-library/`

[15] D. Hunter, "Why you should use io_uring for network I/O", *Red Hat Developer*, 12 April 2023. `https://developers.redhat.com/articles/2023/04/12/why-you-should-use-iouring-network-io`

[16] B. Gregg, *BPF Performance Tools*, Addison-Wesley, 2019.

[17] L. Rice, *Learning eBPF*, O'Reilly Media, 2023.

[18] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, O'Reilly Media, 2017, Chapter 4, "Encoding and Evolution".

[19] M. Kleppmann, "Schema evolution in Avro, Protocol Buffers and Thrift", 5 December 2012. `https://martin.kleppmann.com/2012/12/05/schema-evolution-in-avro-protocol-buffers-thrift.html`

[20] Google, *Protocol Buffers Documentation*. `https://protobuf.dev/`

[21] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm", *USENIX Annual Technical Conference*, 2014.

[22] A. Das, I. Gupta and A. Motivala, "SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol", *IEEE DSN*, 2002.

[23] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek and H. Balakrishnan, "Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications", *ACM SIGCOMM*, 2001.

[24] M. J. Fischer, N. A. Lynch and M. S. Paterson, "Impossibility of Distributed Consensus with One Faulty Process", *Journal of the ACM*, 32(2), 1985.

[25] S. Gilbert and N. Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-tolerant Web Services", *ACM SIGACT News*, 33(2), 2002.

[26] L. Lamport, "The Part-Time Parliament", *ACM Transactions on Computer Systems*, 16(2), 1998.

[27] P. Hunt, M. Konar, F. P. Junqueira and B. Reed, "ZooKeeper: Wait-free Coordination for Internet-scale Systems", *USENIX ATC*, 2010.

[28] J. Kreps, N. Narkhede and J. Rao, "Kafka: a Distributed Messaging System for Log Processing", *NetDB*, 2011.

[29] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine and D. Lewin, "Consistent Hashing and Random Trees", *ACM STOC*, 1997.